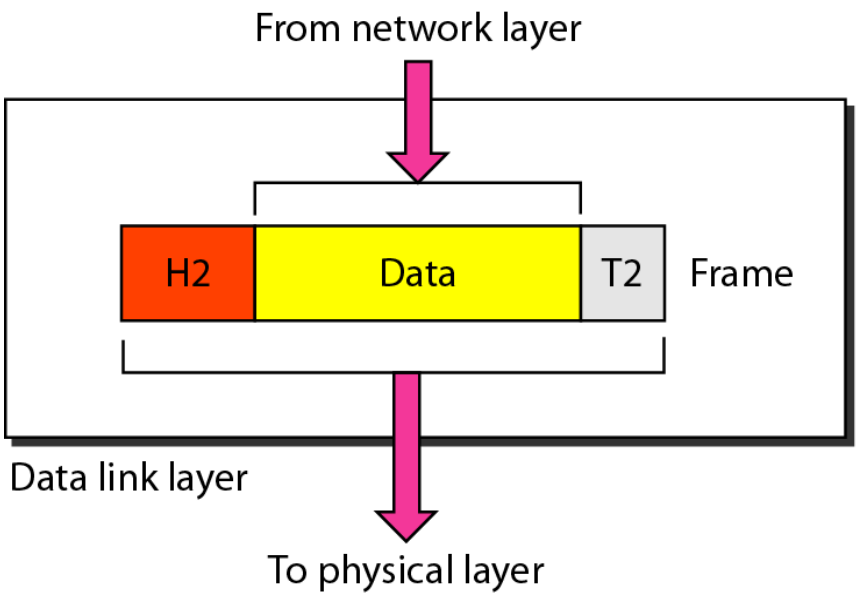
CSE3155 DATA COMMUNICATION AND COMPUTER NETWORKS

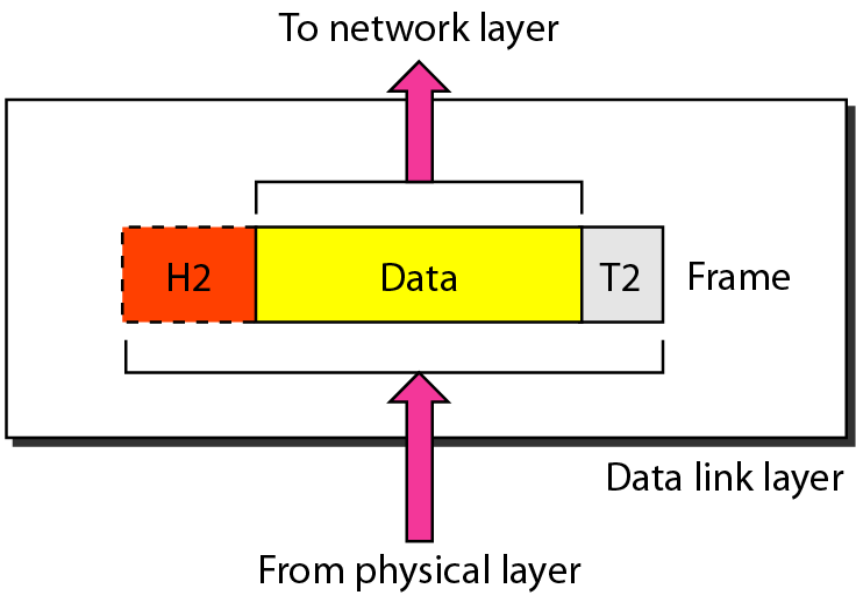
Module-2

Data Link Layer

Overview of Data Link Layer

The data link layer transforms the physical layer, a raw transmission facility, to a link responsible for node-to-node (hop-to-hop) communication. Specific responsibilities of the data link layer include *framing, addressing, flow control, error control, and media access control.*

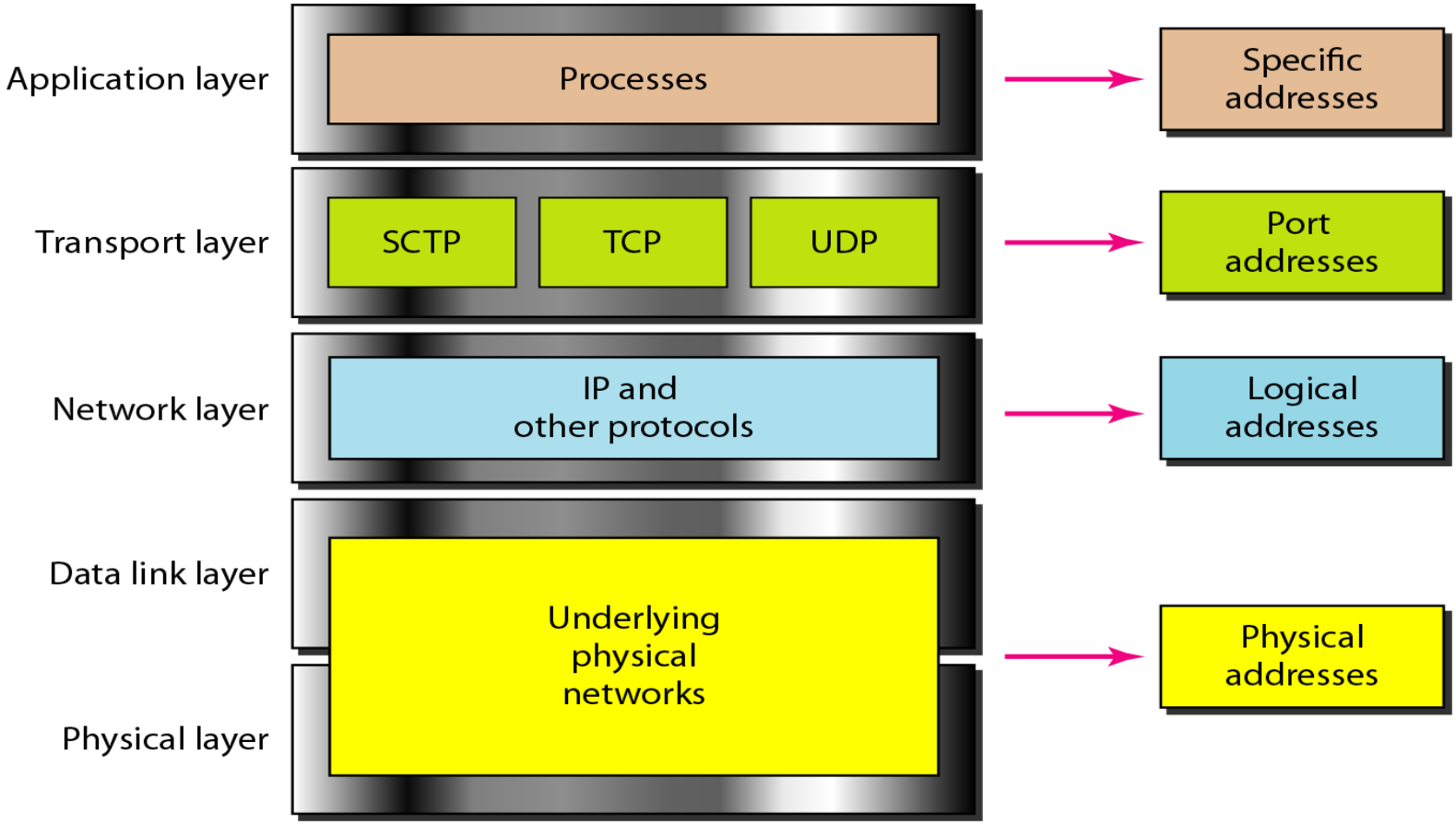




▪Functions of Data Link Layer

1. **Framing:** It divides the stream of bits received from network layer into manageable data units called frames.
2. **Physical Addressing:** It adds a header to the frame to define the sender and receiver of the frame.
3. **Flow Control:** The data link layer imposes a flow control mechanism to avoid overwhelming the receiver.
4. **Error Control**
   - It adds reliability by adding mechanisms to detect and retransmit damaged or lost frames.
   - It also uses a mechanism to recognize duplicate frames by adding the trailer to the end of the frame
5. **Access Control**
   - It determines which device has control over the link at any given time when two or more devices are connected to the same link.

Figure: Relationship *of layers and addresses in TCP/IP*



▪ Sub-layers of Data Link Layer

1. **Logical Link Control (LLC) Layer**
   - The Logical Link Control layer controls frame synchronization, flow control and error checking.
2. **Media Access Control (MAC) Layer**
   - The MAC layer is responsible for moving data packets to and from one Network Interface Card (NIC) to another across a shared channel.

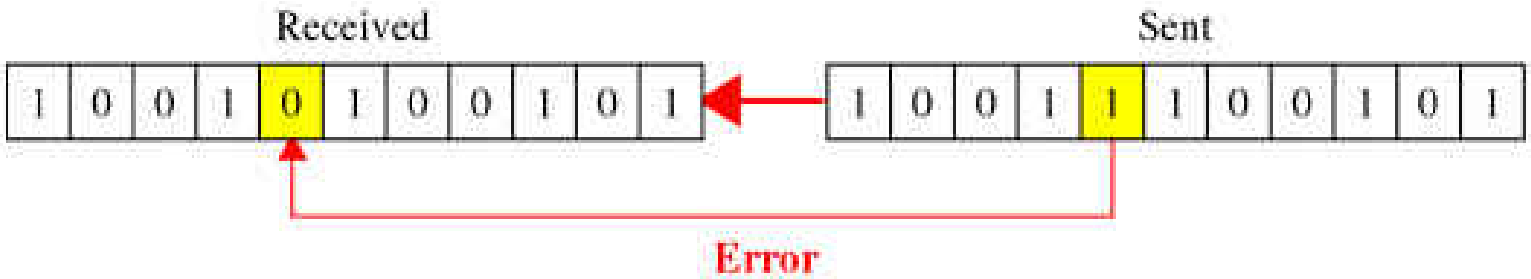
Error Control

- Networks must be able to transfer data from one device to another with acceptable accuracy. For most applications, a system must guarantee that the data received are identical to the data transmitted.
- Any time data are transmitted from one node to the next, they can become corrupted in passage. Many factors can alter one or more bits of a message.
- Some applications require a mechanism for detecting and correcting **errors.**
- Some applications can tolerate a small level of error. For example, random errors in audio or video transmissions may be tolerable, but when we transfer text, we expect a very high level of accuracy.

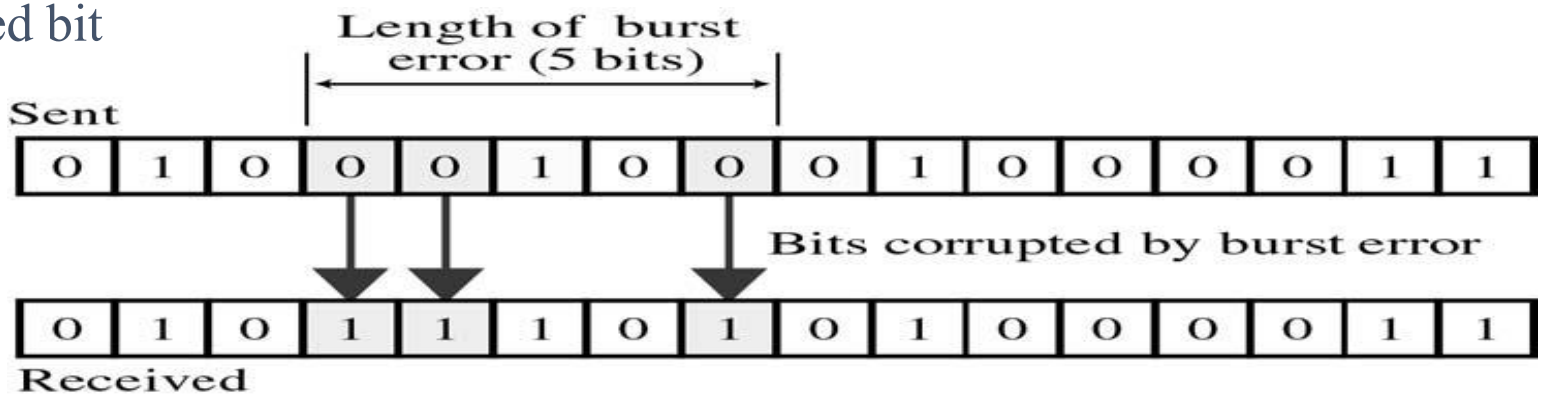
Types of Errors

There are two main types of errors in transmissions:

1. **Single bit error** : It means only one bit of data unit is changed from 1 to 0 or from 0 to 1.



2. **Burst error** : It means two or more bits in data unit are changed from 1 to 0 from 0 to 1. In burst error, it is not necessary that only consecutive bits are changed. The length of burst error is measured from first changed bit to last changed bit

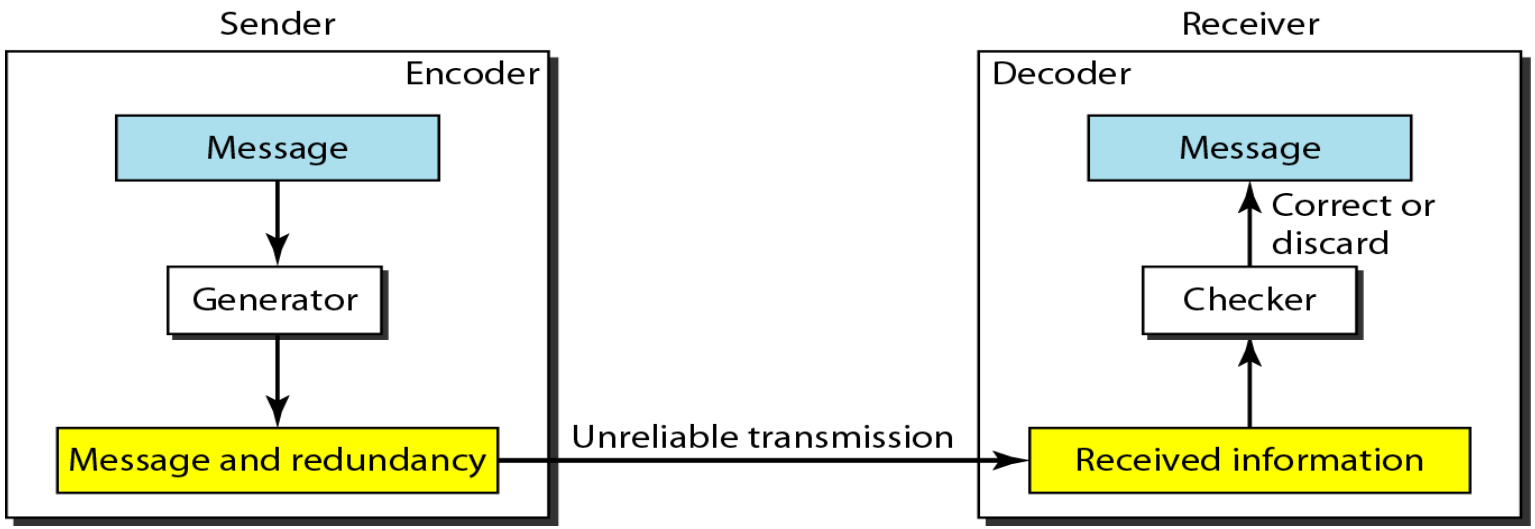


Error Detection vs Error Correction

There are two types of attacks against errors:

Error Detecting Codes: Include enough redundancy bits to *detect* errors and use ACKs and retransmissions to recover from the errors.

Error Correcting Codes: Include enough redundancy to detect *and* correct errors. The use of error-correcting codes is often referred to as forward error correction.



Cntd…

Redundancy

- The central concept in detecting or correcting errors is **redundancy**. To be able to detect or correct errors, we need to send some extra bits with our data.
- These **redundant bits** are added by the sender and removed by the receiver. Their presence allows the receiver to detect or correct corrupted bits.

Detection versus Correction

- The correction of errors is more difficult than the detection. In error detection, we are looking only to see if any error has occurred. The answer is a simple yes or no. We are not even interested in the number of errors. A single-bit error is the same for us as a burst error.
- In error correction, we need to know the exact number of bits that are corrupted and more importantly, **their location** in the message. The number of the errors and the size of the message are important factors.

Cntd…

Forward Error Correction versus Retransmisson

- There are two main methods of error correction. Forward error correction is the process in which the **receiver tries to guess the message by using redundant bits**. This is possible, as we see later, if the number of errors is small.
- Correction by retransmission is a technique in which the receiver detects the occurrence of an error **and asks the sender to resend the message**. Resending is repeated until a message arrives that the receiver believes is error-free

Cntd…

Coding

- Redundancy is achieved through various **coding schemes**. The sender adds redundant bits through a process that creates a relationship between the redundant bits and the actual data bits.
- The receiver checks the relationships between the two sets of bits to detect or correct the errors. The ratio of redundant bits to the data bits and the robustness of the process are important factors in any coding scheme.
- Figure shows the general idea of coding.

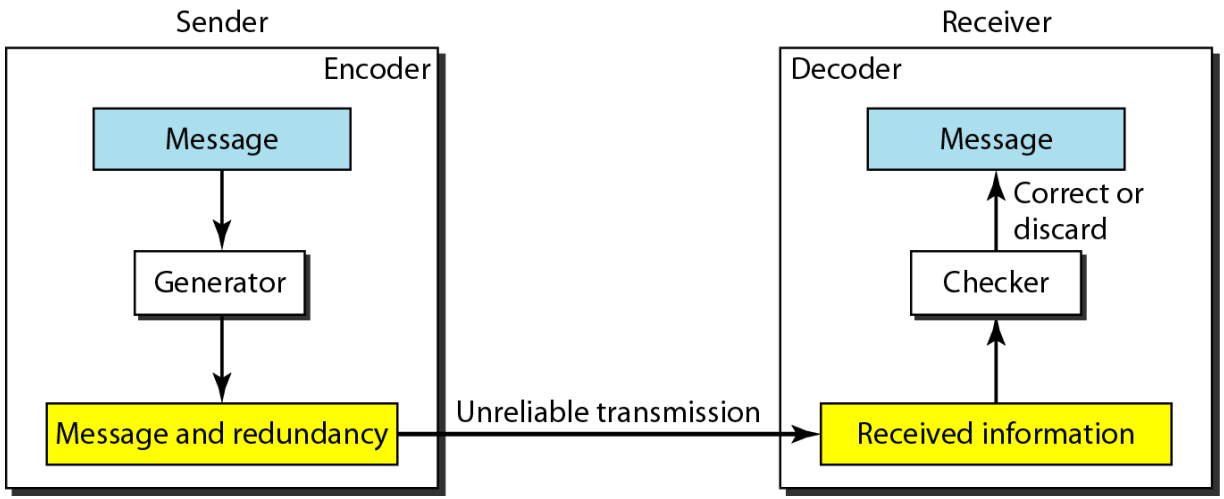


Figure: *The structure of encoder and decoder*

Block Coding

- In block coding, we divide our message into blocks, each of k bits, called **datawords**. We add r redundant bits to each block to make the length n = k + r. The resulting n-bit blocks are called **codewords**.
- **How the extra r bits is chosen or calculated** is something we will discuss later.
- **For the moment, it is important to know that we have a set of datawords, each of size k, and a set of codewords, each of size of n.**
- With k bits, we can create a combination of 2^k datawords; with n bits, we can create a combination of 2^n codewords. Since n > k, the number of possible codewords is larger than the number of possible datawords.

HAMMING DISTANCE

- The Hamming distance between two words is the number of differences between corresponding bits.

- The minimum Hamming distance is the smallest Hamming distance between all possible pairs in a set of words.

We can count the number of 1s in the Xoring of two words

1. *The Hamming distance d(000, 011) is 2 because*

$$000 \oplus 011 \text{ is } 011 \text{ (two 1s)}$$

2. *The Hamming distance d(10101, 11110) is 3 because*

$$10101 \oplus 11110 \text{ is } 01011 \text{ (three 1s)}$$

Find the minimum Hamming distance of the coding scheme

Datawords	*Codewords*
00	000
01	011
10	101
11	110

Solution

We first find all Hamming distances.

$d(000, 011) = 2$ $d(000, 101) = 2$ $d(000, 110) = 2$ $d(011, 101) = 2$
$d(011, 110) = 2$ $d(101, 110) = 2$

The d_{min} in this case is 2.

Find the minimum Hamming distance of the coding scheme

Dataword	*Codeword*
00	00000
01	01011
10	10101
11	11110

Solution

We first find all the Hamming distances.

$d(00000, 01011) = 3$	$d(00000, 10101) = 3$	$d(00000, 11110) = 4$
$d(01011, 10101) = 4$	$d(01011, 11110) = 3$	$d(10101, 11110) = 3$

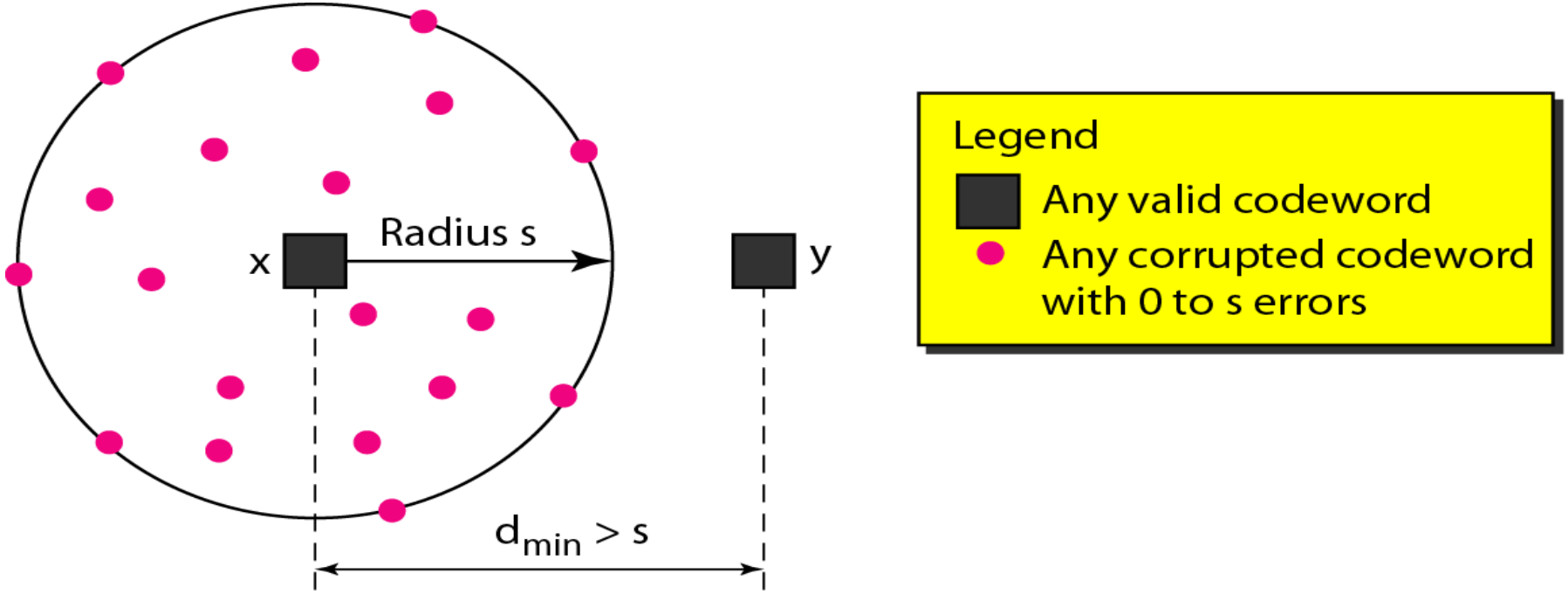
The d_{min} in this case is 3.

Minimum Distance for Error Detection

- In a set of codewords, the minimum Hamming distance is the smallest Hamming distance between all possible pairs of codewords.

- To guarantee the detection of up to s errors in all cases, the minimum Hamming distance in a block code must be dmin = s + 1.

Geometric concept for finding d_{min} in error detection



Example

A code scheme has a Hamming distance $d_{min} = 4$. What is the error detection of this scheme?

Solution

This code guarantees the detection of up to three errors ($s = 3$).

Some Linear Block Codes

- Let us now study some linear block codes. These codes are trivial because we can easily find the encoding and decoding algorithms and check their performances.

Simple Parity-Check Code

- Perhaps the most familiar error-detecting code is the **simple parity-check code**. In this code, a k-bit dataword is changed to an n-bit codeword where n = k + 1.
- The extra bit, called the parity bit, is selected to make the total number of 1s in the codeword even.
- Although some implementations specify an odd number of 1s, we discuss the even case.

Cntd…

- Our first code (Table 1) is a parity-check code with k = 2 and n =3. The code in Table 3 is also a parity-check code with k =4 and n =5.

Datawords	*Codewords*	*Datawords*	*Codewords*
0000	00000	1000	10001
0001	00011	1001	10010
0010	00101	1010	10100
0011	00110	1011	10111
0100	01001	1100	11000
0101	01010	1101	11011
0110	01100	1110	11101
0111	01111	1111	11110

Table: ***Simple parity-check code C(5, 4)***

Cntd…

- Figure below shows a possible structure of an encoder (at the sender) and a decoder (at the receiver).

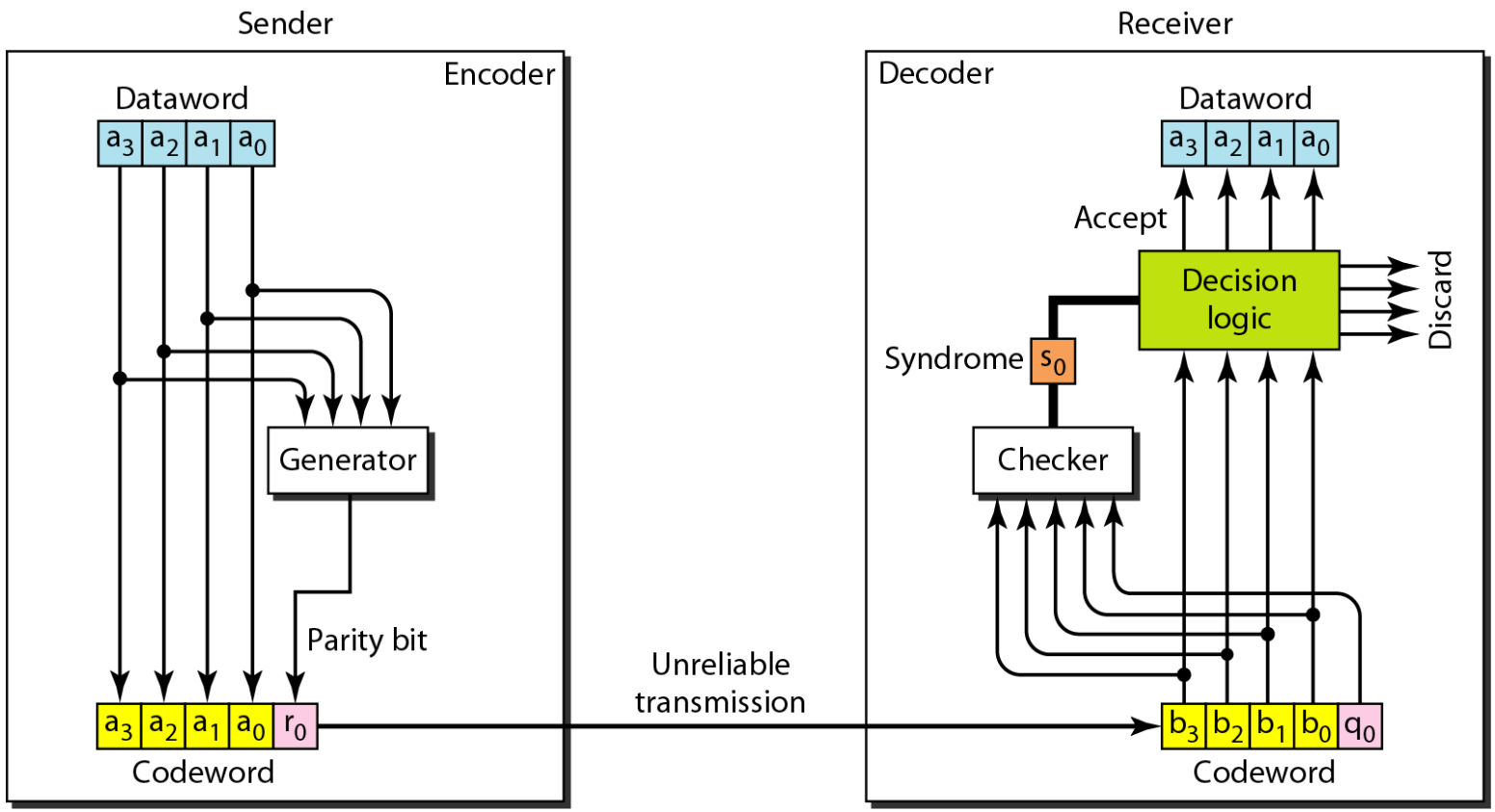


Figure: ***Encoder and decoder for simple parity-check code***

Cntd…

- At the sender, the generator takes a copy of a 4-bit dataword (a_0, a_1, a_2, and a_3) and generates a parity bit r_0. The dataword and the **parity bit** create the 5-bit codeword.
- The **parity bit** that is added **makes the number of 1s in the codeword even**.
- This is normally done by adding the 4 bits of the dataword (modulo-2); the result is the parity bit. In other words,

$$r_0 = a_3 + a_2 + a_1 + a_0 \text{ (modulo-2)}$$

- If the number of 1s is even, the result is 0; if the number of 1s is odd, the result is 1. In both cases, the total number of 1s in the codeword is even.
- The sender sends the codeword which may be corrupted during transmission. The receiver receives a 5-bit word.
- The checker at the receiver does the same thing as the generator in the sender with one exception: **The addition is done over all 5 bits**.

Cntd…

- The result, which is called the **syndrome**, is just 1 bit. The syndrome is 0 when the number of 1s in the received codeword is even; otherwise, it is 1.

$$s_0 = b_3 + b_2 + b_1 + b_0 + q_0 \text{ (modulo-2)}$$

- The syndrome is passed to the decision logic analyzer.
- If the **syndrome is 0**, there is no error in the received codeword; the data portion of the received codeword is accepted as the dataword.
- If the **syndrome is 1**, the data portion of the received codeword is discarded. The dataword is not created.

Example

- Let us look at some transmission scenarios. Assume the sender sends the dataword 1011. The codeword created from this dataword is 10111, which is sent to the receiver. We examine five cases:

1. No error occurs; the received codeword is 10111. The syndrome is 0. **The dataword 1011 is created.**
2. One single-bit error changes a_1. The received codeword is 10011. The syndrome is 1. **No dataword is created**.
3. One single-bit error changes r_0. The received codeword is 10110. The syndrome is 1. **No dataword is created.** Note that although none of the dataword bits are corrupted, no dataword is created because the code is not sophisticated enough to show the position of the corrupted bit.

Cntd…

4. An error changes r_0 and a second error changes a_3 .The received codeword is 00110. The syndrome is 0. The dataword 0011 is created at the receiver. Note that here **the dataword is wrongly created due to the syndrome value**. The simple parity-check decoder cannot detect an **even number of errors.**

5. Three bits—a_3, a_2, and a_1—are changed by errors. The received codeword is 01011. The syndrome is 1.The dataword is not created. This shows that the simple parity check, **guaranteed to detect one single error, can also find any odd number of errors.**

CYCLIC CODES

Cyclic codes *are special linear block codes with one extra property. In a cyclic code, if a codeword is cyclically shifted (rotated), the result is another codeword.*

Topics discussed in this section:

- **Cyclic Redundancy Check**
- **Polynomials**
- **Hardware Implémentation**
- **Advantages of Cyclic Codes**

Cyclic Codes

- **Cyclic codes** are special linear block codes with one extra property. In a cyclic code, if a codeword is **cyclically shifted** (rotated), the result is another codeword.
- For example, if 1011000 is a codeword and we cyclically left-shift, then 0110001 is also a codeword.

Cyclic Redundancy Check

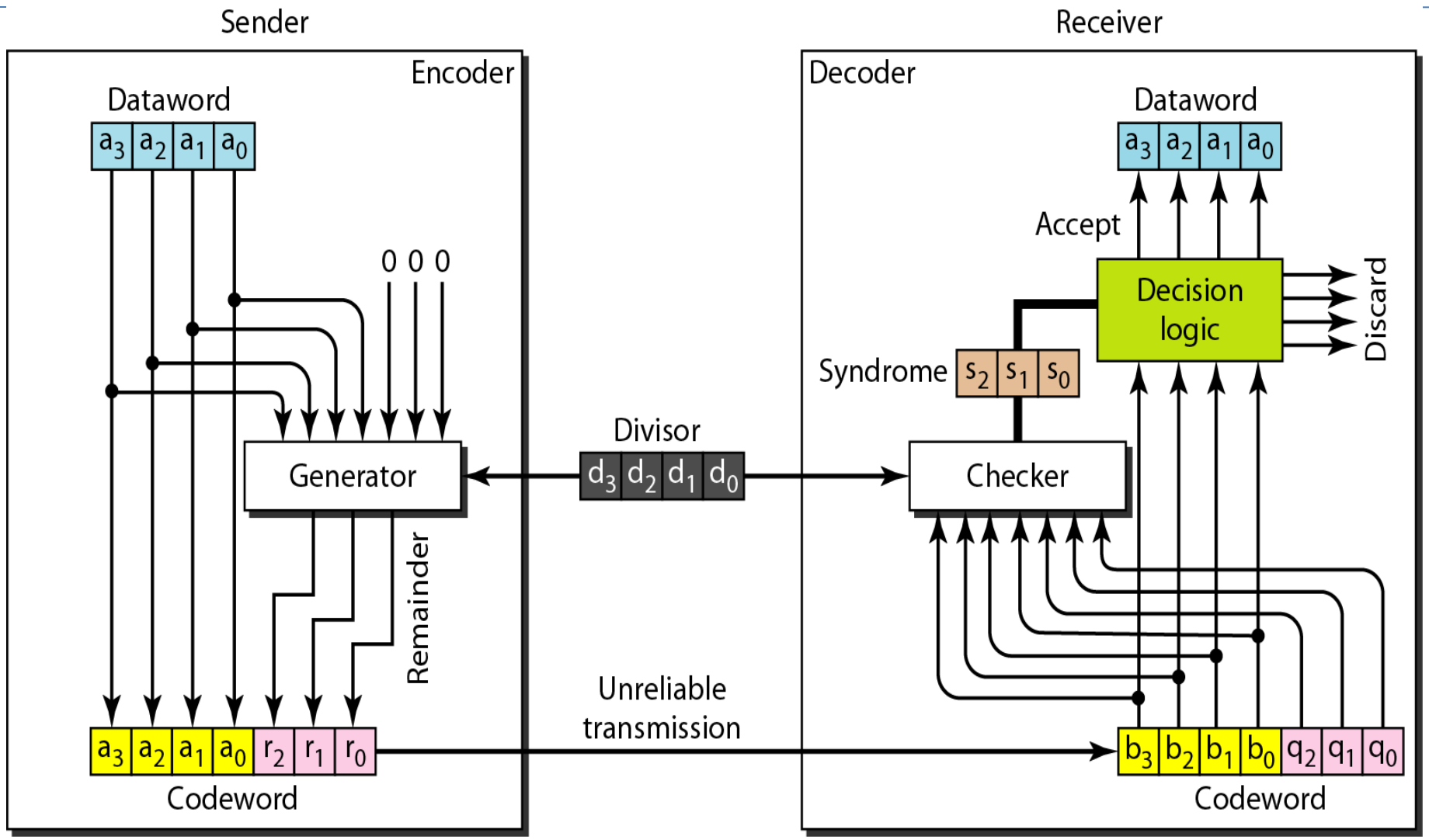
- We can create cyclic codes to **detect errors**.
- In this section, we simply discuss a category of cyclic codes called the **cyclic redundancy check** (CRC) that is used in networks such as LANs and WANs.

Cntd…

Table *A CRC code with C(7, 4)*

Dataword	*Codeword*	*Dataword*	*Codeword*
0000	0000000	1000	1000101
0001	0001011	1001	1001110
0010	0010110	1010	1010011
0011	0011101	1011	1011000
0100	0100111	1100	1100010
0101	0101100	1101	1101001
0110	0110001	1110	1110100
0111	0111010	1111	1111111

CRC encoder and decoder



Cntd…

- In the encoder, the dataword has k bits (4 here); the codeword has n bits (7 here). The size of the dataword is **augmented by adding n - k (3 here) 0s to the right-hand side of the word**.
- The n-bit result is fed into the generator. The **generator uses a divisor of size n - k + 1 (4 here), predefined and agreed upon.**
- The generator divides the augmented dataword by the divisor (modulo-2 division).
- The quotient of the division is discarded; the remainder ($r_2 r_1 r_0$) is appended to the dataword to create the **codeword.**
- The decoder receives the possibly corrupted codeword. A copy of all n bits is fed to the checker which is a replica of the generator.
- The **remainder produced by the checker is a syndrome of n - k (3 here) bits,** which is fed to the decision logic analyzer.
- The analyzer has a simple function. **If the syndrome bits are all 0s**, the 4 leftmost bits of the **codeword are accepted as the dataword** (interpreted as no error); **otherwise, the 4 bits are discarded (error).**

Encoder

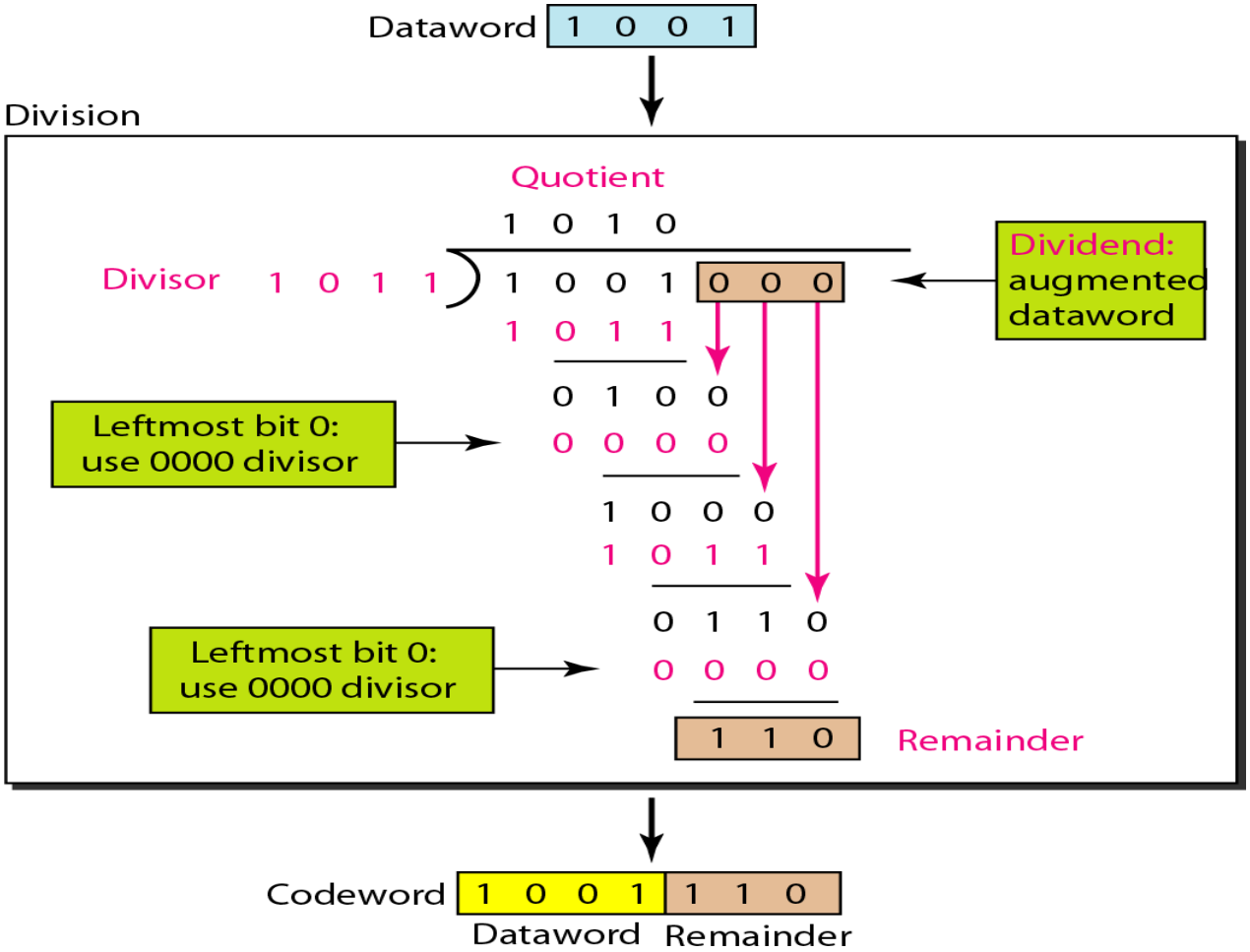


Figure: ***Division in CRC encoder***

Decoder

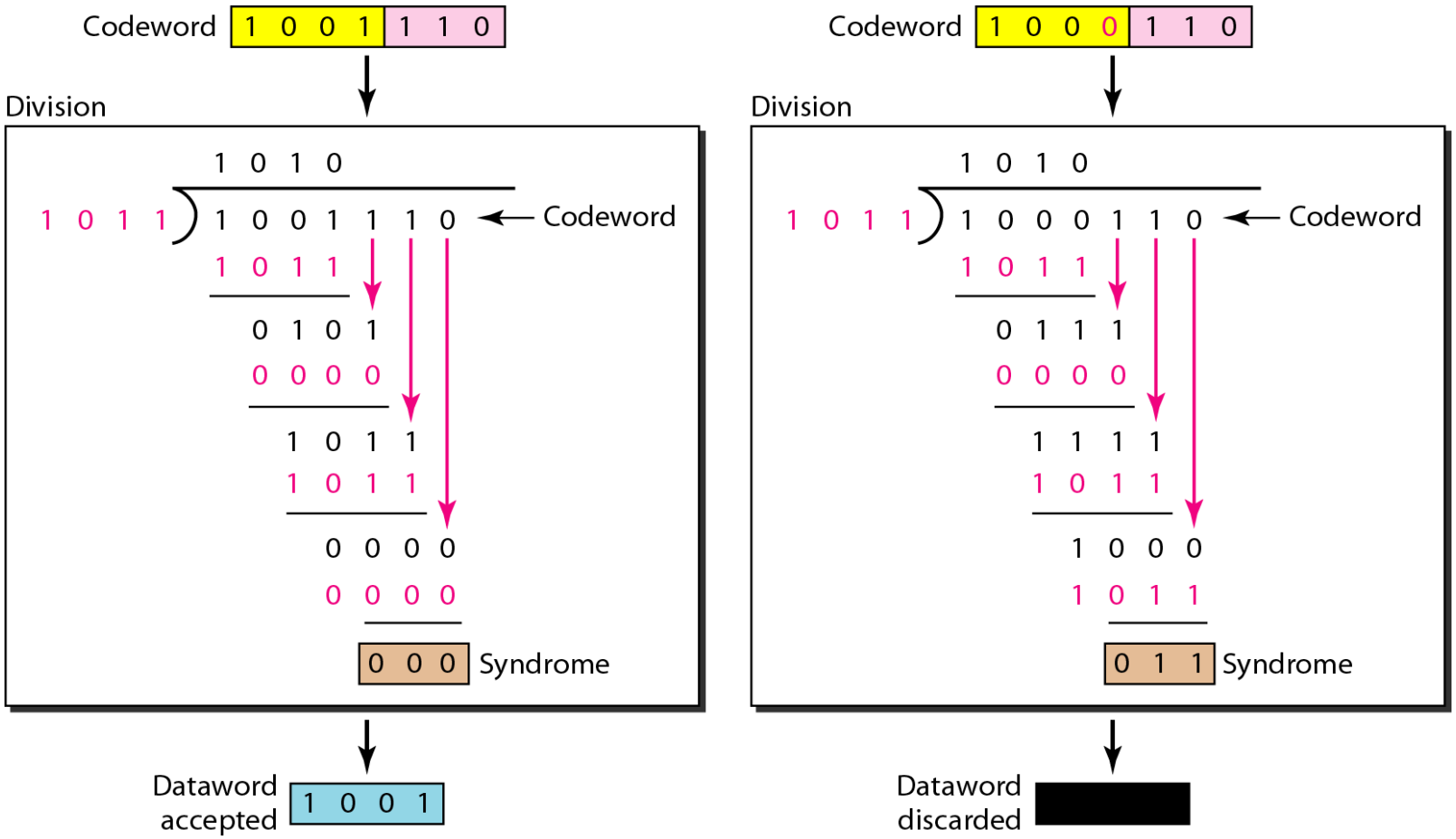
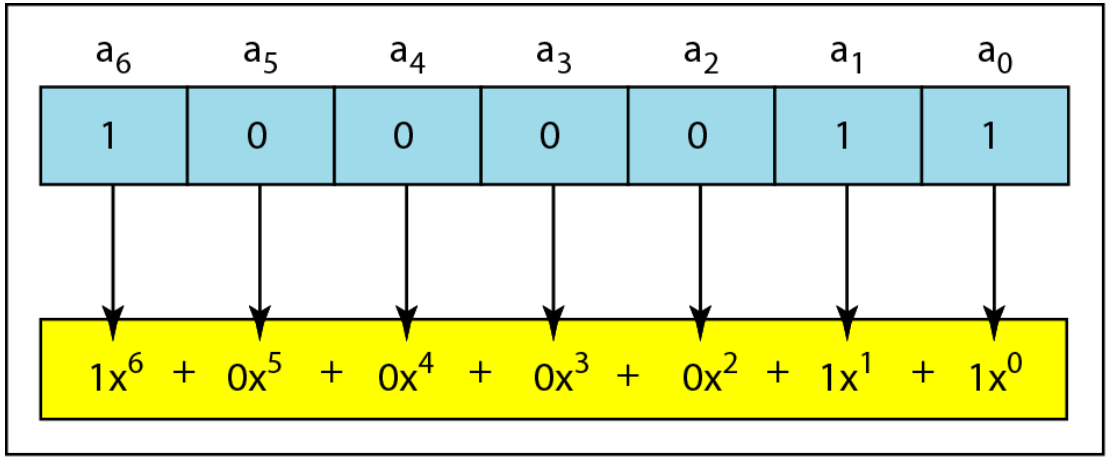


Figure: ***Division in the CRC decoder for two cases***

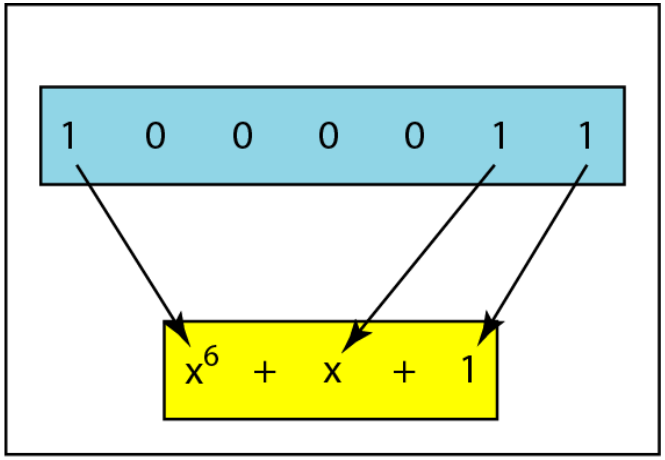
Cntd…

- The codeword can change during transmission. The decoder does the same division process as the encoder.
- The remainder of the division is the **syndrome**. If the syndrome is all 0s, there is no error; the dataword is separated from the received codeword and accepted.
- Otherwise, everything is discarded.
- Figure below shows two cases: The left hand figure shows the value of syndrome when no error has occurred; the syndrome is 000.
- The right-hand part of the figure shows the case in which there is one single error. The syndrome is not all 0s (it is 011)

A polynomial to represent a binary word



a. Binary pattern and polynomial



b. Short form

Advantages of Cyclic Codes

- We have seen that cyclic codes have a very good performance in **detecting** single-bit errors, double errors, an odd number of errors, and burst errors.
- They can easily be implemented in hardware and software. They are especially fast when implemented in hardware.
- This has made cyclic codes a good candidate for many networks.

FLOW CONTROL

- **Flow control** is a set of procedures that tells the sender how much data it can transmit before it must wait for an acknowledgement from the receiver.
- Any receiving device has a limited speed at which it can process incoming data and a limited amount of memory to store the incoming data.
- Incoming data must be checked and processed before they can be used.
- The rate of such processing is slower than the rate of transmission.
- For this reason, each receiving device has a block of memory, called a buffer, reserved for storing incoming data until they are processed.
- If the buffer begins to fill up, the receiver must be able to tell the sender to halt transmission until it is once again able to receive.

FLOW CONTROL

Flow control is technique that a transmitting entity does not conquer a receiving entity with data.

Two functional mechanisms are acknowledgment and timeouts.

- After getting each frame, the receiver will send ACK to sender.
- If the sender does not receive ACK up to reasonable amount of time, the it retransmit the original frame; waiting for reasonable amount of time is called **timeout**.
- The general strategy of using acknowledgments and timeouts to implement reliable delivery is sometimes called **automatic repeat request (ARQ)**(To be discussed in transport layer)

The two flow control mechanisms are:

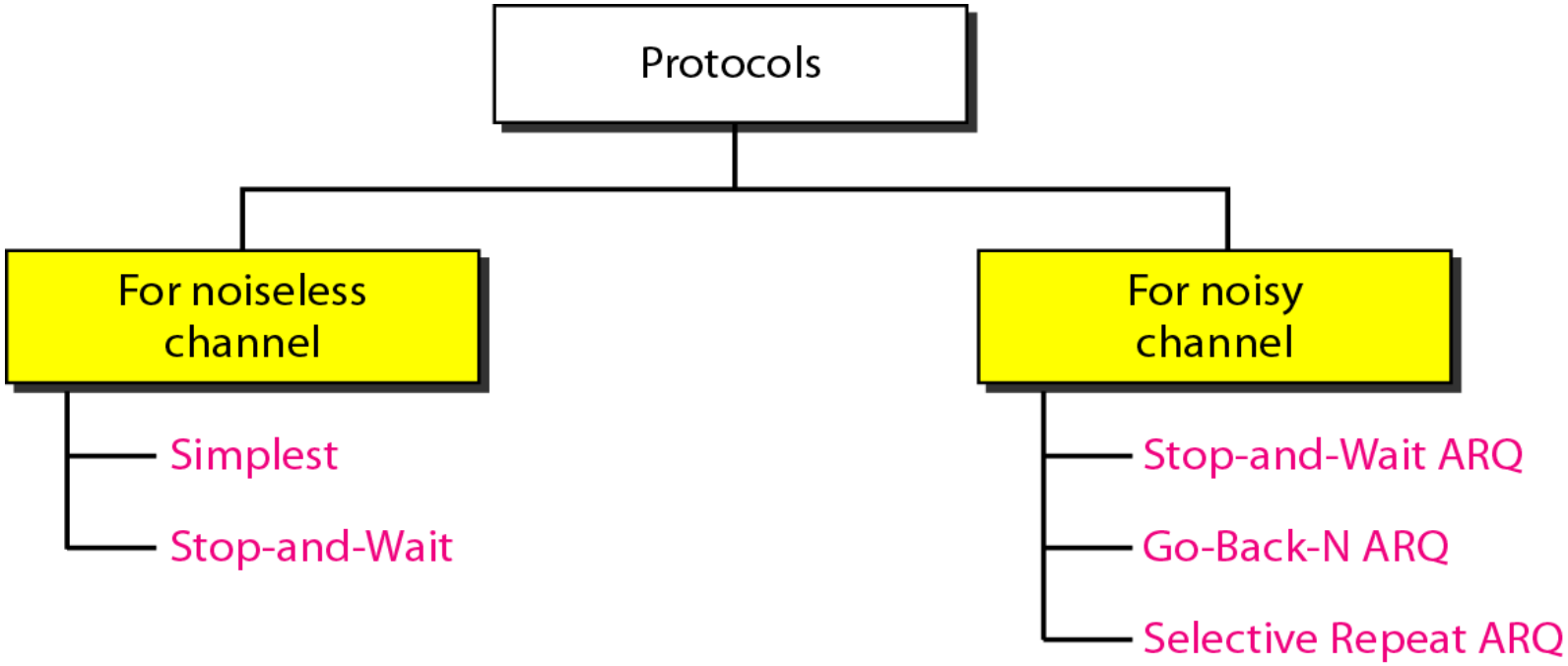
Simple and Stop and Wait Flow Control

Sliding Window Flow Control(To be discussed in transport layer)

Data Link Protocols

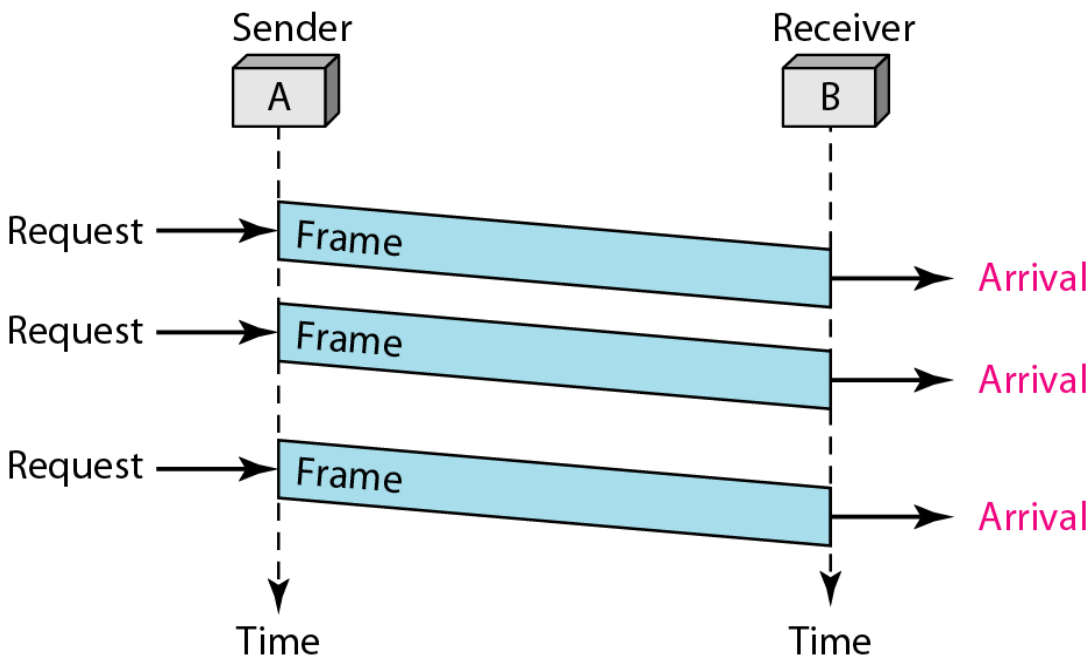
The protocols are normally implemented in software by using one of the common programming languages

- A Simple Protocol
- A Stop-and-Wait Protocol



A simple protocol

- In order to appreciate the step by step development of efficient and complex protocols we will begin with a simple but unrealistic protocol. In this protocol: Data are transmitted in one direction only
- The transmitting (Tx) and receiving (Rx) hosts are always ready
- Processing time can be ignored
- Infinite buffer space is available
- No errors occur; i.e. no damaged frames and no lost frames (perfect channel)

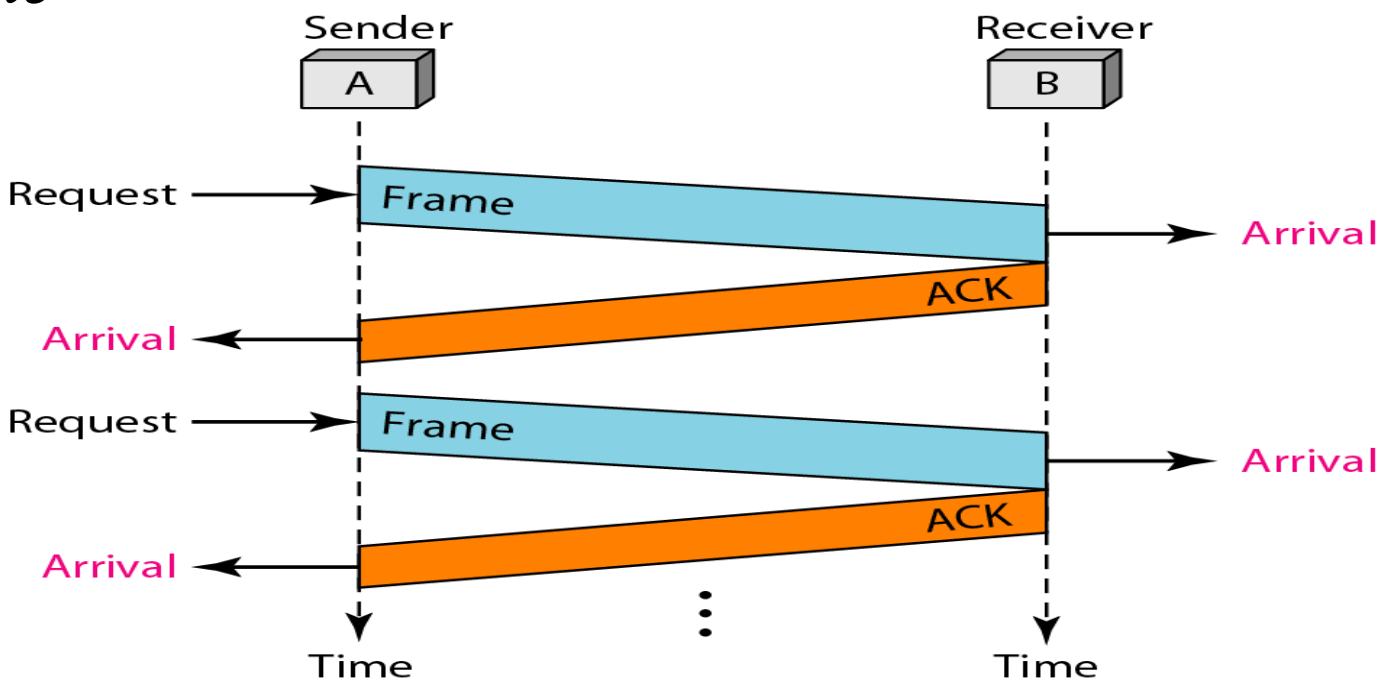


A stop-and-wait protocol

- In this protocol we assume that Data are transmitted in one direction only
- No errors occur (perfect channel)
- The receiver can only process the received information at a finite rate
- These assumptions imply that the transmitter cannot send frames at a rate faster than the receiver can process them.
- The problem here is how to prevent the sender from flooding the receiver.
- A general solution to this problem is to have the receiver provide some sort of feedback to the sender. The process could be as follows: The receiver send an acknowledge frame back to the sender telling the sender that the last received frame has been processed and passed to the host; permission to send the next frame is granted. The sender, after having sent a frame, must wait for the acknowledge frame from the receiver before sending another frame.
    - **This protocol is known as *stop-and-wait*.**

Stop & Wait Protocol

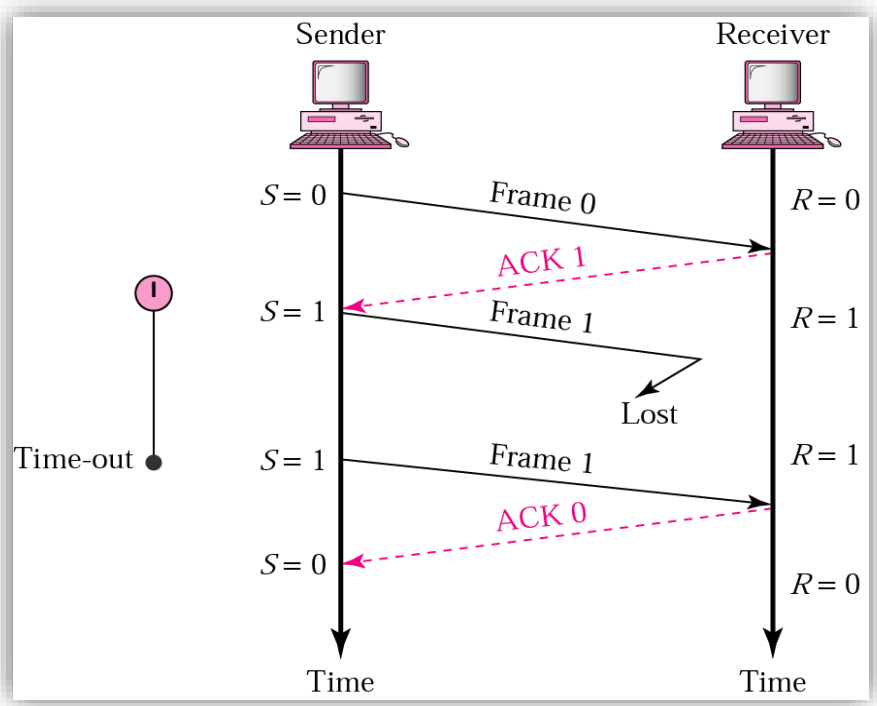
The sender sends one frame and waits for feedback from the receiver. When the ACK arrives, the sender sends the next frame



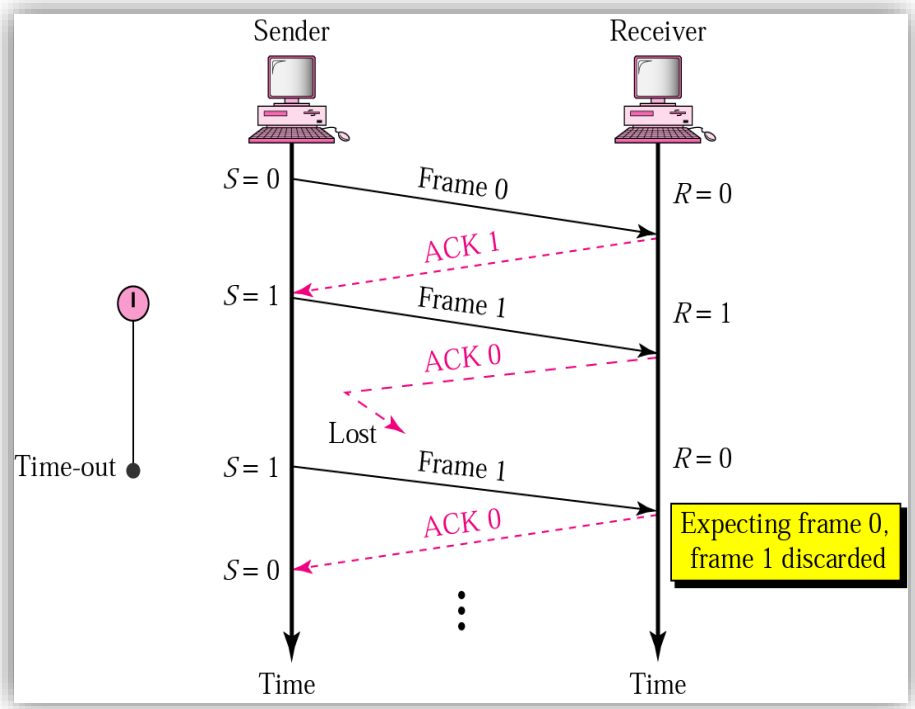
STOP AND WAIT PROTOCOL

Imagine a situation where the receiver has just sent an ACK frame back to the sender saying that it correctly received and already passed a frame to its host. However, the ACK frame gets lost completely, the sender times out and retransmits the frame. There is no way for the receiver to tell whether this frame is a retransmitted frame or a new frame, so the receiver accepts this duplicate happily and transfers it to the host. The protocol thus fails in this aspect.

STOP-AND-WAIT, LOST FRAME



Stop-and-Wait, lost ACK frame



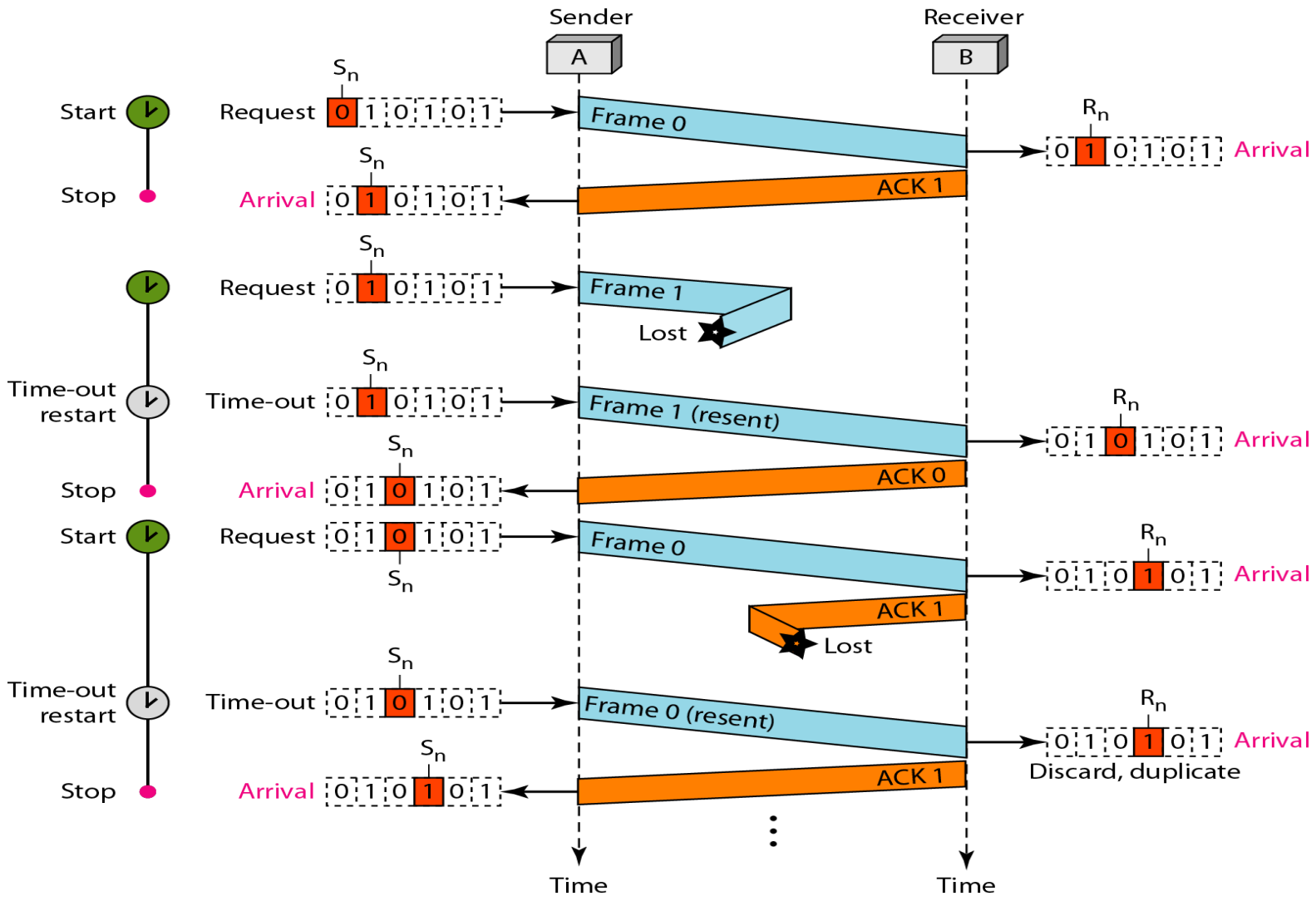
STOP AND WAIT PROTOCOL

- To overcome this problem it is required that the receiver be able to distinguish a frame that it is seeing for the first time from a retransmission.
- One way to achieve this is to have the sender put a **sequence number** in the header of each frame it sends.
- The receiver then can check the sequence number of each arriving frame to see if it is a new frame or a duplicate to be discarded.

The receiver needs to distinguish only 2 possibilities:

- a new frame or a duplicate; a **1-bit sequence number** is sufficient. At any instant the receiver expects a particular sequence number. Any wrong sequence numbered frame arriving at the receiver is rejected as a duplicate.
- A correctly numbered frame arriving at the receiver is accepted, passed to the host, and the expected sequence number is incremented by 1 (modulo 2).

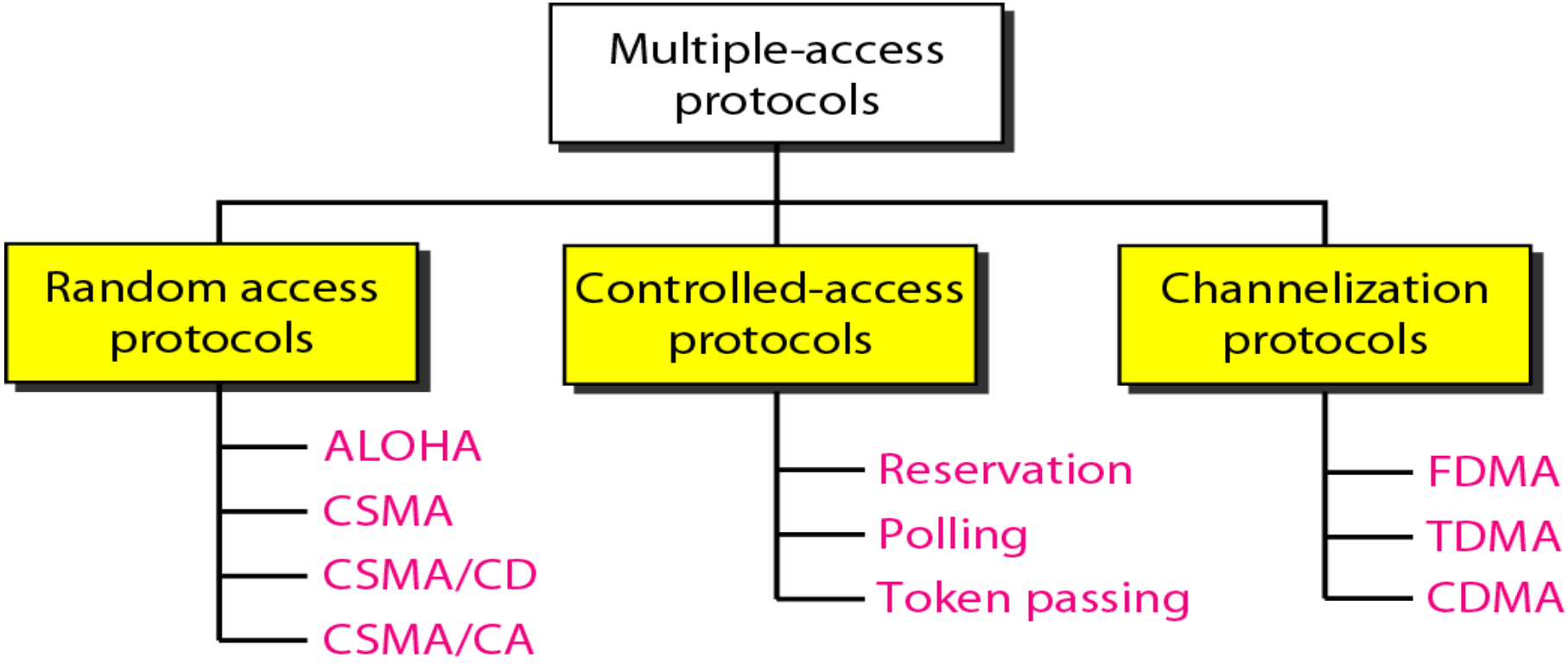
Flow Diagram of a Stop & Wait Protocol



MULTIPLE ACCESS PROTOCOL

- *The medium access control (MAC) is a sublayer of the data link layer.*
- *The MAC sublayer emulates a full-duplex logical communication channel in a multipoint network.*
- *This channel may provide unicast, multicast, or broadcast communication service.*
- *The MAC sublayer uses MAC protocols to ensure that signals sent from different stations across the same channel don't collide. Eg: two people speak*
- *A multiple-access protocol to coordinate access to the link (multipoint or broadcast link).*
- *Many protocols have been devised to handle access to a shared link.*

Taxonomy of Multiple-access protocols



Random Access Protocols

- Also called **contention-based** access
- No station is **superior** to another station and No station is assigned to control another.
- A station that has data to send uses a procedure defined by the protocol to make a **decision on whether or not to send**.
- Decision depends on the state of the medium (**idle or busy**).
- Two features of RAP:
  - **No scheduled time for a station to transmit**
  - **No rules specify which station should send next**
- Stations compete with one another to access the medium

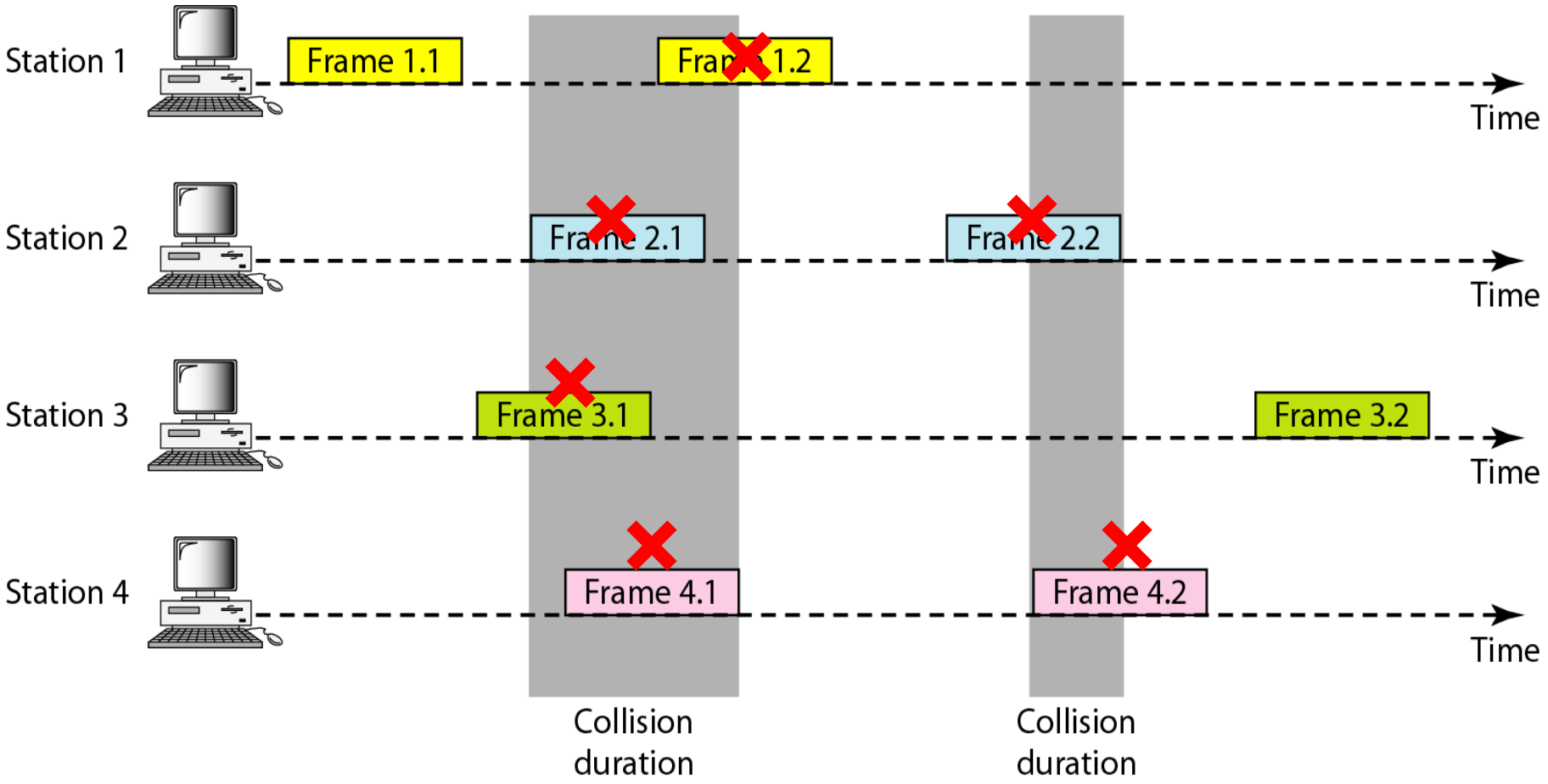
ALOHA

- Aloha is the type of Random access protocol
- **ALOHA,** was developed at the University of Hawaii in early 1970.
- It was designed for a radio (wireless) LAN, but it can be used on any shared medium.
- It has two types :one is **Pure Aloha** and another is **Slotted Aloha**.
- There is a potential of **collisions**
- The **medium is shared** between the stations.
- When a station **sends data**, another station may attempt to do so at the **same time**.
- The data from the two stations **collide** and become garbled.

Pure ALOHA

- The original ALOHA protocol is called ***pure ALOHA.***
- This is a **simple** but elegant protocol.
- The idea is that each station sends a frame whenever it has a **frame to send** (multiple access).
- However, since there is only **one channel to share**, there is the **possibility of collision** between frames from different stations.

Frames in Pure ALOHA



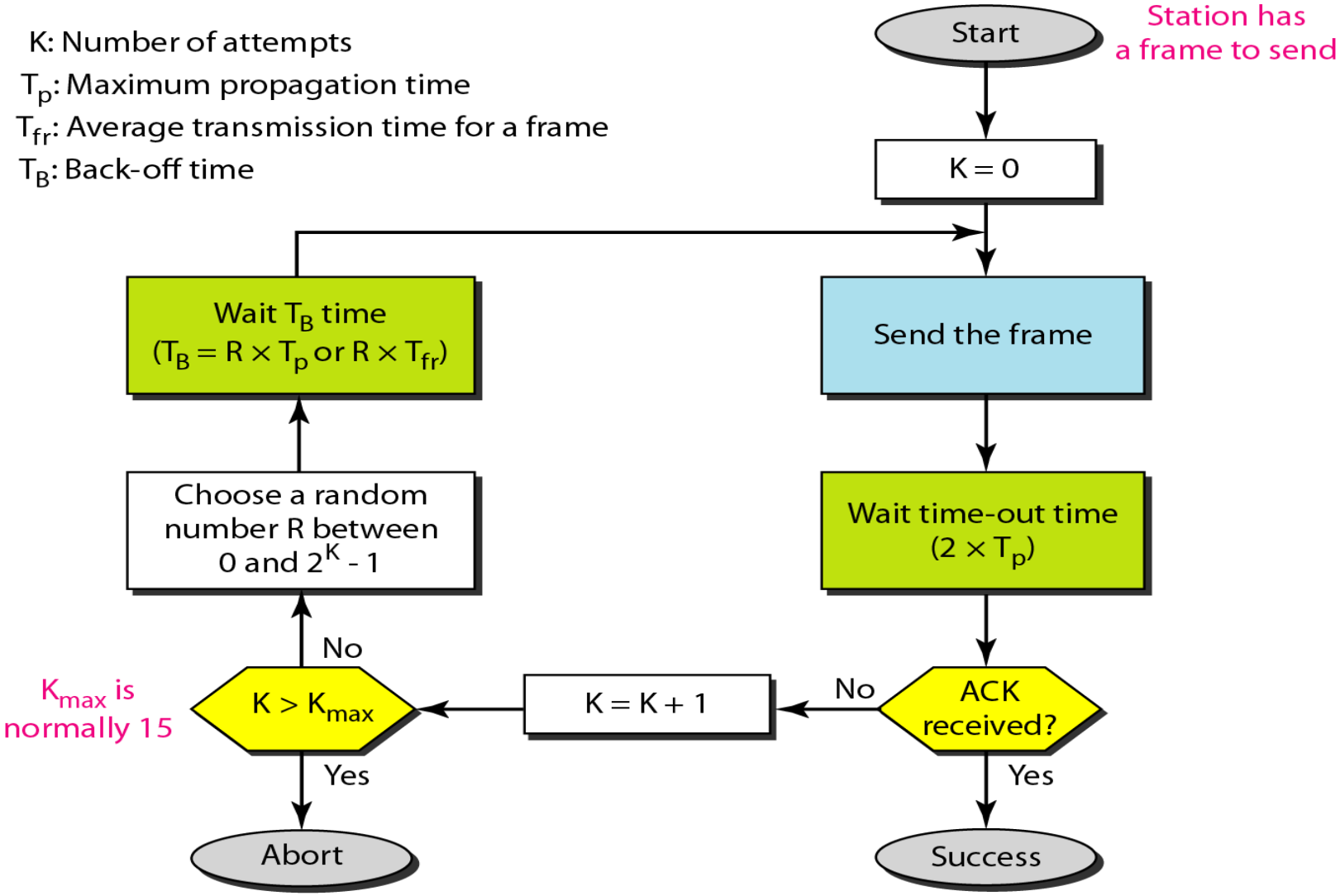
Procedure for pure ALOHA protocol

K: Number of attempts

T_p: Maximum propagation time

T_{fr}: Average transmission time for a frame

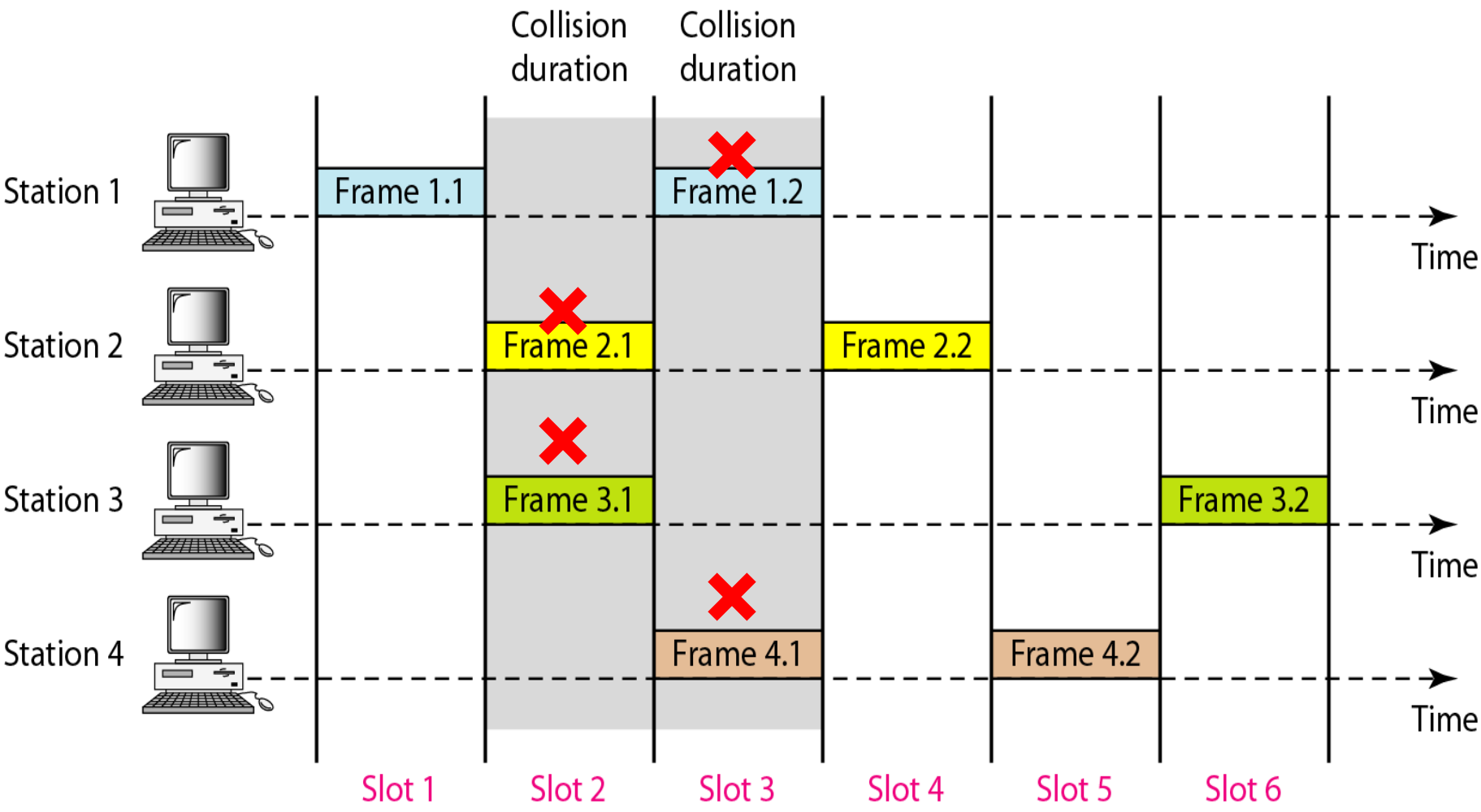
T_B: Back-off time



Slotted ALOHA

- Slotted Aloha divides the time of shared channel into discrete intervals called as **time slots**.
- Any station can transmit its data in any time slot.
- The only condition is that station must start its transmission from the beginning of the time slot.
- If the beginning of the slot is missed, then station has to wait until the beginning of the next time slot.
- A **collision** may occur if two or more stations try to transmit data at the beginning of the same time slot.
- Slotted ALOHA was invented to **improve** the efficiency of pure ALOHA.

Slotted ALOHA



CSMA(Carrier Sense Multiple Access)

- *To minimize the chance of collision and to increase the performance*
- *Principle of CSMA: "sense before transmit" or "listen before talk"*
- *Carrier busy= Transmission is taking place*
- *Carrier idle= No transmission currently taking place*
- *CSMA can reduce the possibility of collision, but it cannot eliminate it.*

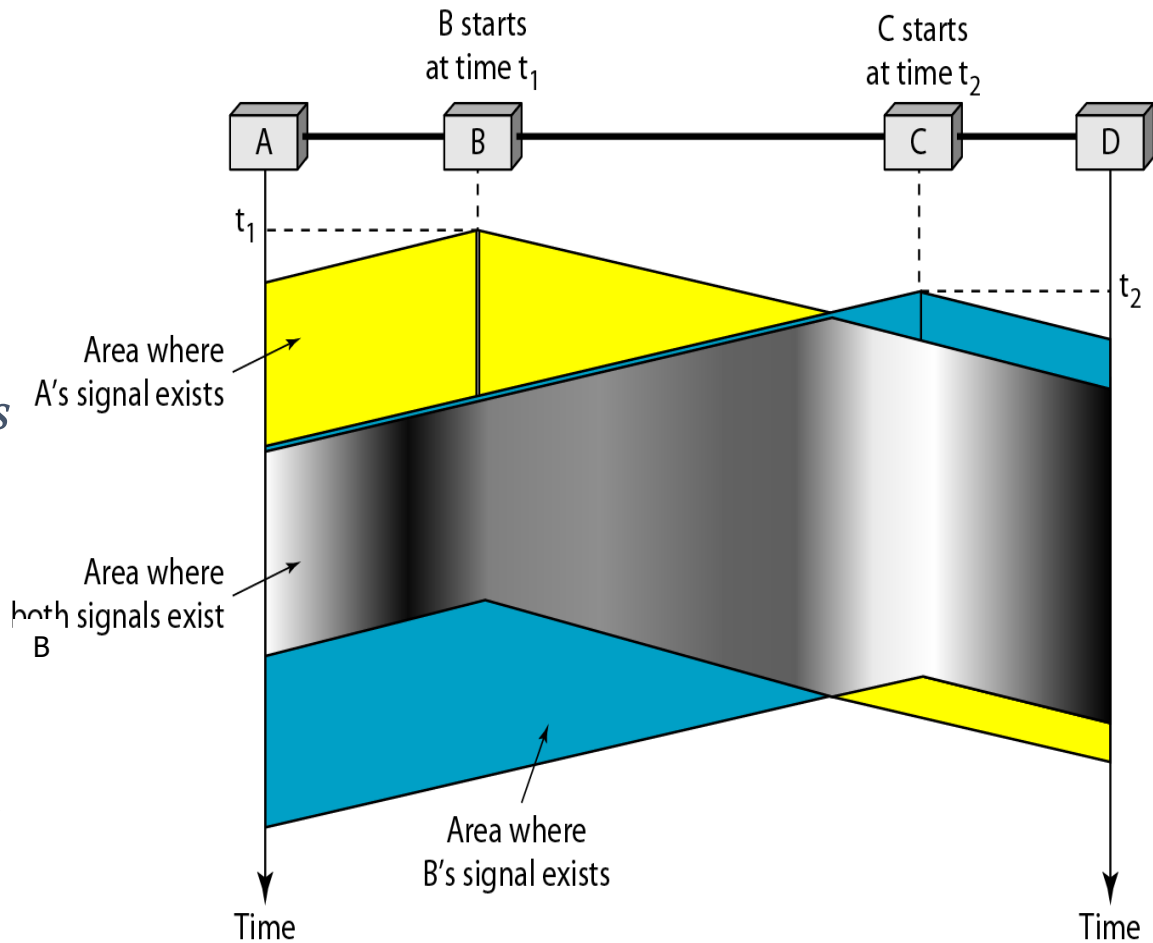
Collision in CSMA

At time t1, station B senses the medium and finds it idle, so it sends a frame.

At time t2 (t2 > t1), station C senses the medium and finds it idle because, at this time, the firs bits from station B have not reached station C.

Station C also sends a frame.

The two signals collide and both frames are destroyed.



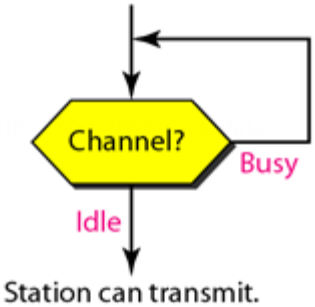
Persistence Methods

- What should a station do if the channel is busy?
- What should a station do if the channel is idle?
- Three methods have been devised to answer these questions:
  - **1-persistent method**
  - **nonpersistent method**
  - ***p*-persistent method**

1-Persistent

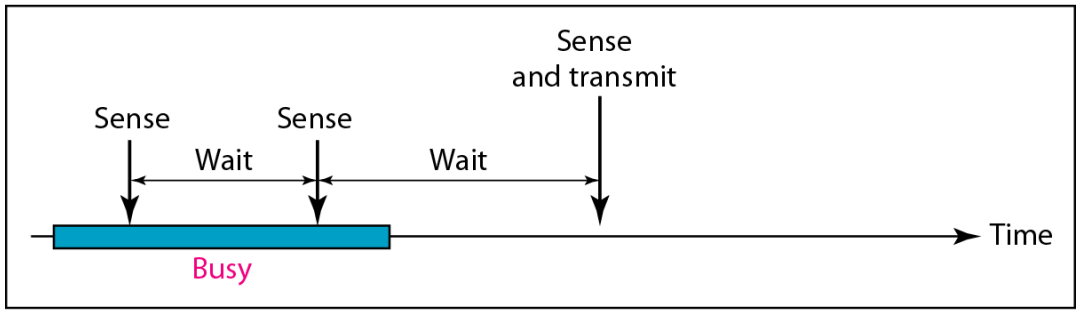
- The ***1-persistent method*** is simple and straightforward.
- In this method, after the **station finds the line idle**, it sends its frame immediately (**with probability 1**).
- This method has the **highest chance of collision** because **two or more stations may find the line idle** and send their frames immediately.

a. 1-persistent

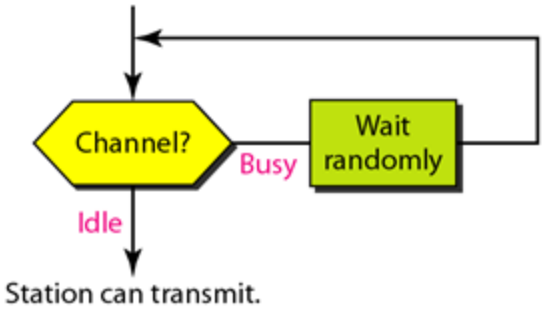


Non-persistent

- In the *non-persistent method,* a station that has a frame to send senses the line
- If the line is idle, it sends immediately.
- If the line is not idle, it waits a **random amount** of time and then senses the line again.
- The nonpersistent approach reduces the chance of collision

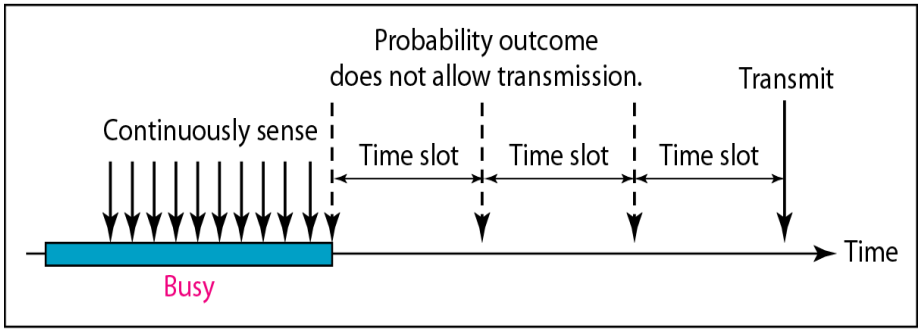


b. Nonpersistent

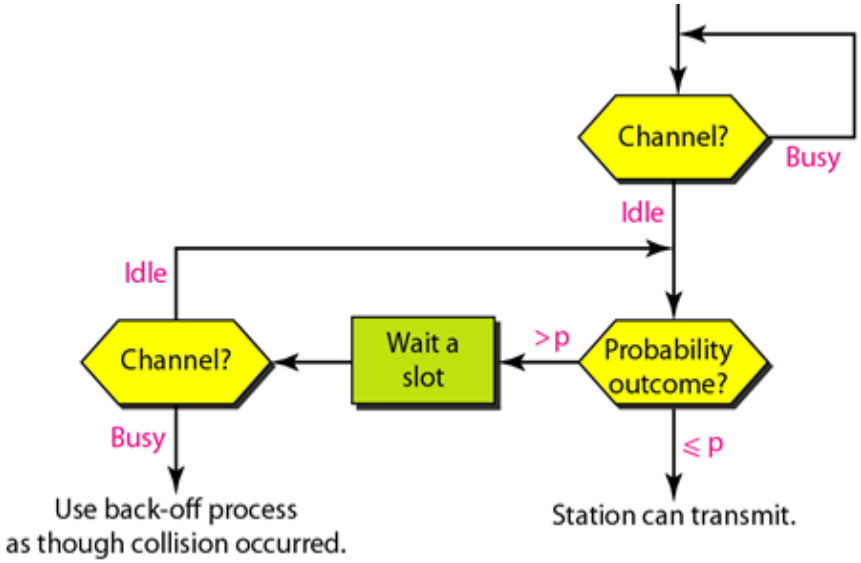


p-Persistent

- *If the channel has time slots with a slot duration equal to or greater than the maximum propagation time.*
- *The p-persistent approach combines the advantages of the other two strategies.*
- *It reduces the chance of collision and improves efficiency.*



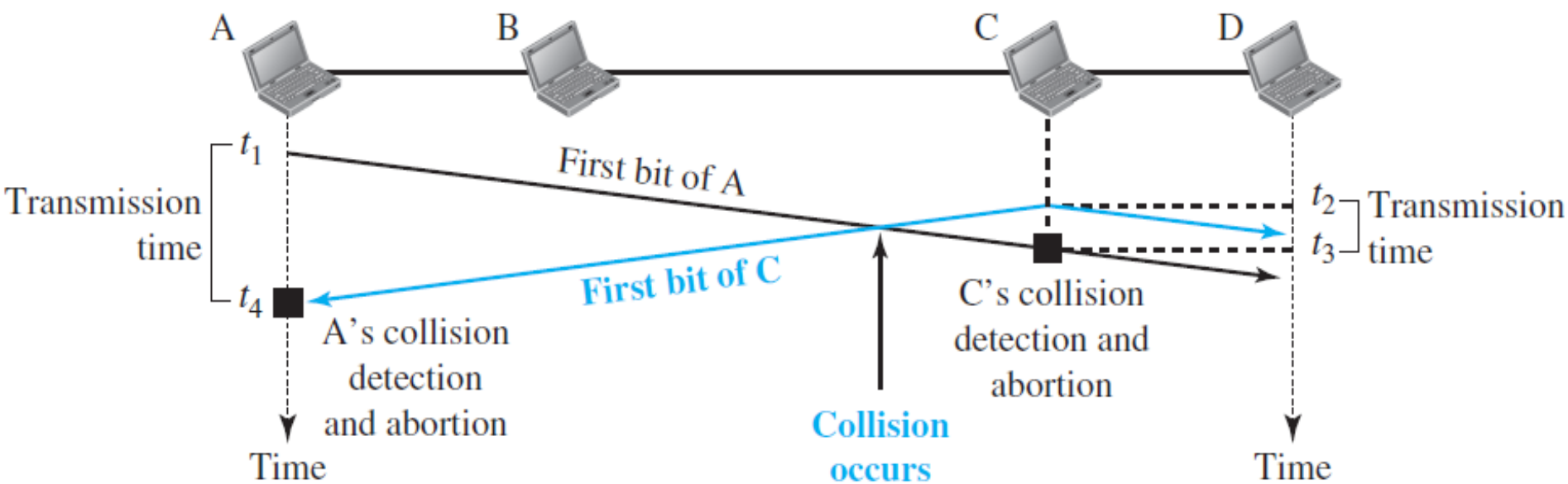
c. p-persistent



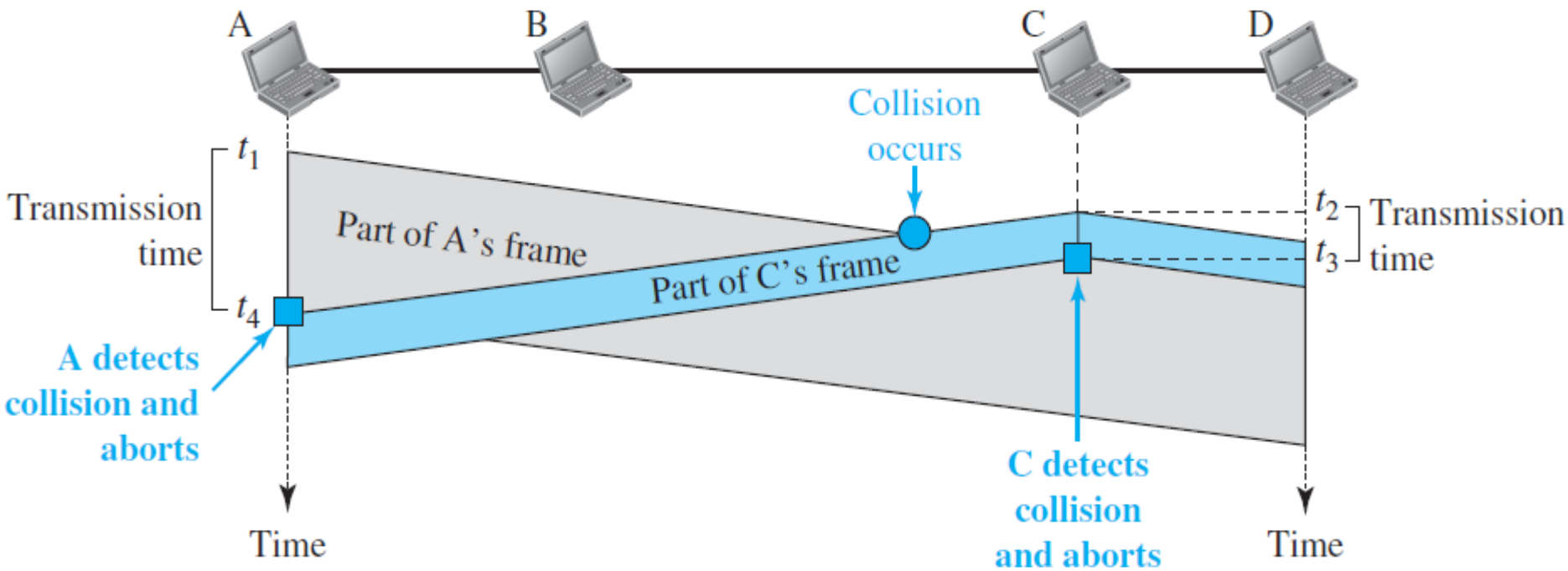
CSMA/CD

- **C**arrier **S**ense **M**ultiple **A**ccess with **C**ollision **D**etection
- Station monitors channel while sending a frame
- If, however, there is a collision, the frame is sent again.
- Eg. Collision of the first bit in CSMA/CD,stations A and C are involved in the collision.

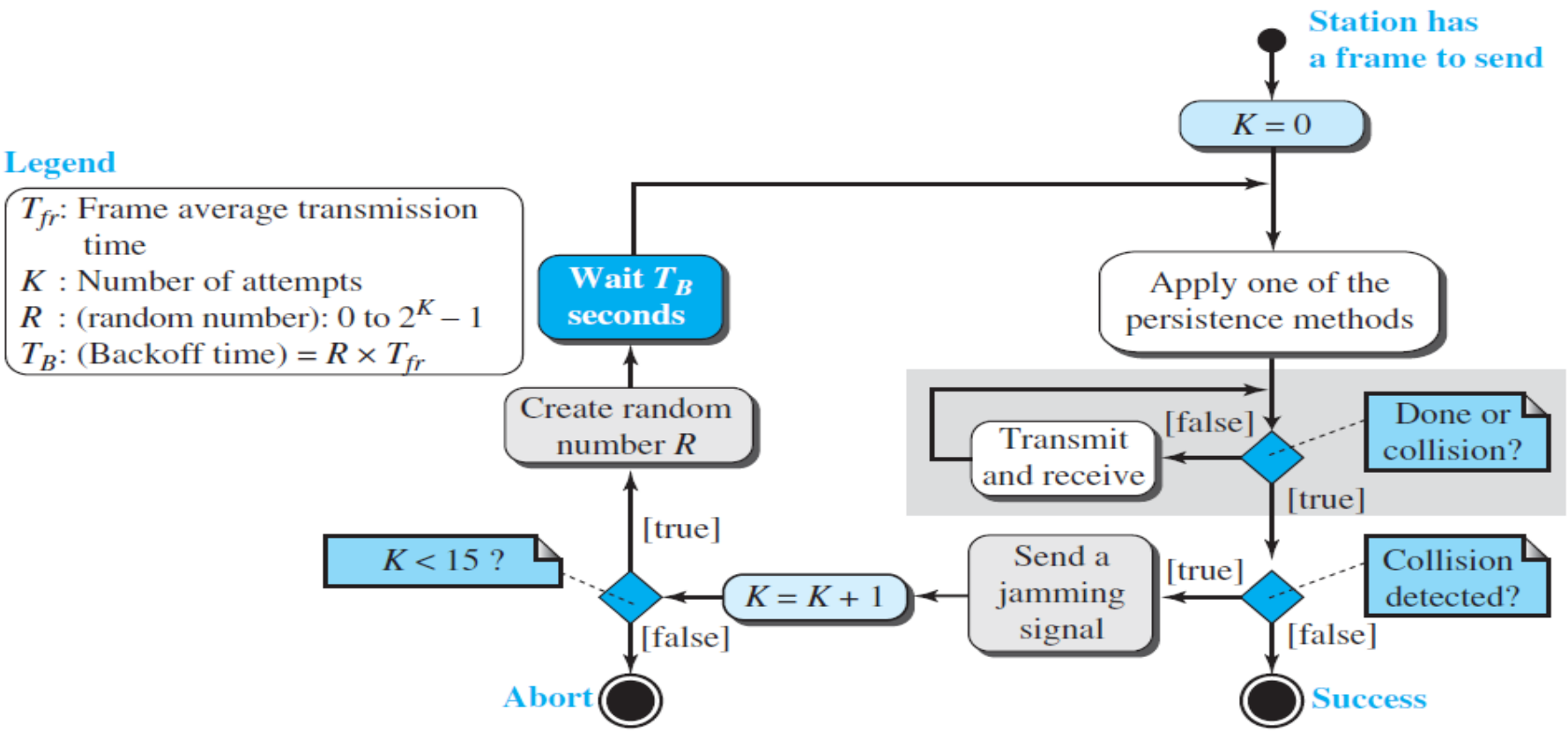
- At time ***t*1**, **station A** has executed its persistence procedure and starts sending the bits of its frame.
- At time ***t*2**, **station C** has **not yet sensed the first bit sent by A.**
- **Station C** executes its persistence procedure and starts sending the bits in its frame, which propagate both to the left and to the right.
- The **collision occurs** sometime after time *t*2.
- Station **C detects a collision at time *t*3** when it receives the first bit of A's frame.
- **Station C** immediately aborts transmission.
- **Station A detects collision at time *t*4** when it receives the first bit of C's frame; it also immediately aborts transmission.
- Looking at the figure, we see that A transmits for the duration ***t*4 − *t*1**; C transmits for the duration ***t*3 − *t*2.**



Collision and abortion in CSMA/CD

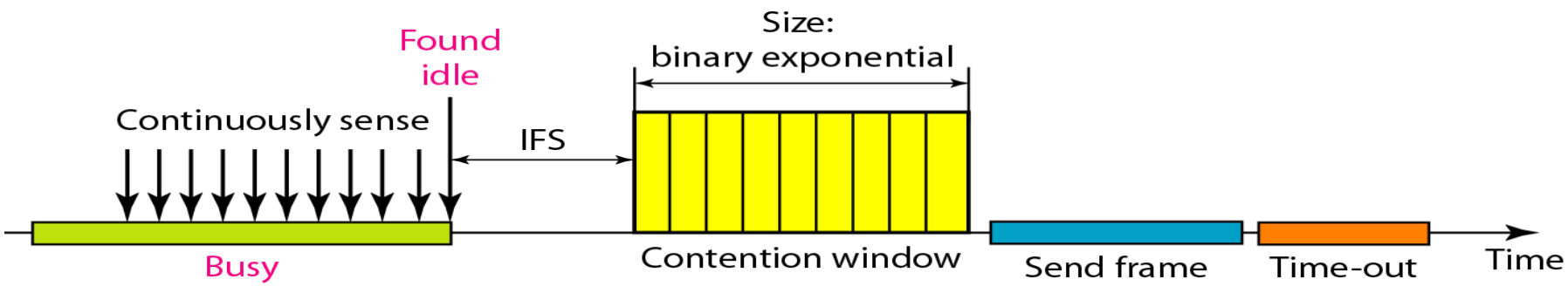


CSMA/CD: Flow Diagram



CSMA/CA

- **C**arrier **S**ense **M**ultiple **A**ccess with **C**ollision **A**voidance was invented for **wireless networks**
- Used in a network where collision cannot be detected
- Collisions are avoided through the use of CSMA/CA's three strategies:
  - **Interframe space (IFS)**
  - **Contention window**
  - **Acknowledgments**



CSMA/CA

Interframe space (IFS)

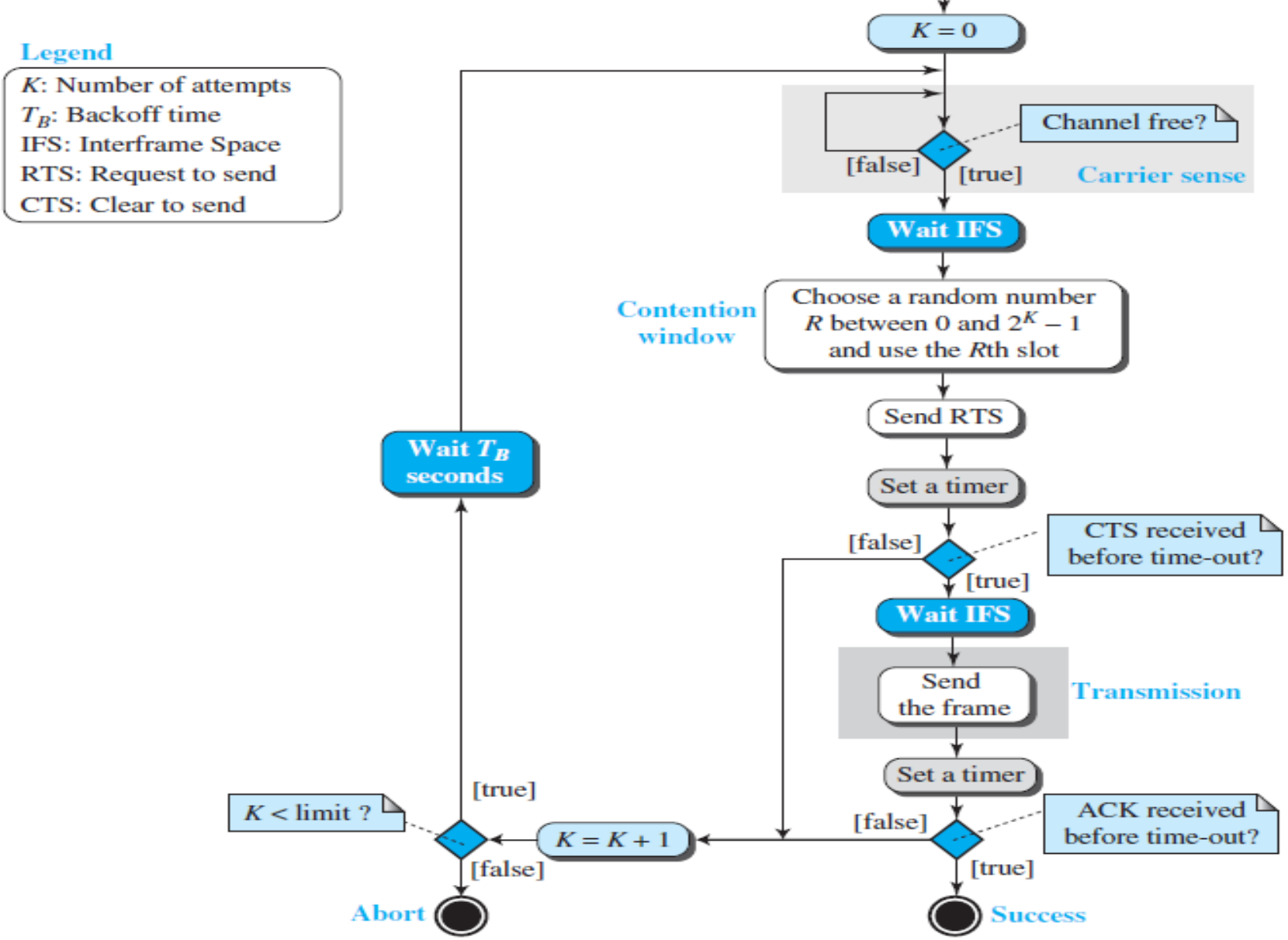
- When an idle channel is found, the station does not send immediately.
- It waits for a period of time called the ***interframe space*** or ***IFS.***

Contention Window

- The **contention window** is an amount of time divided into slots.
- if station determine that the channel is free, they wait a random amount of time before they start sending.
- This time window doubles with each collision and corresponds to the binary exponential backoff (BEB) that is familiar from CSMA/CD.

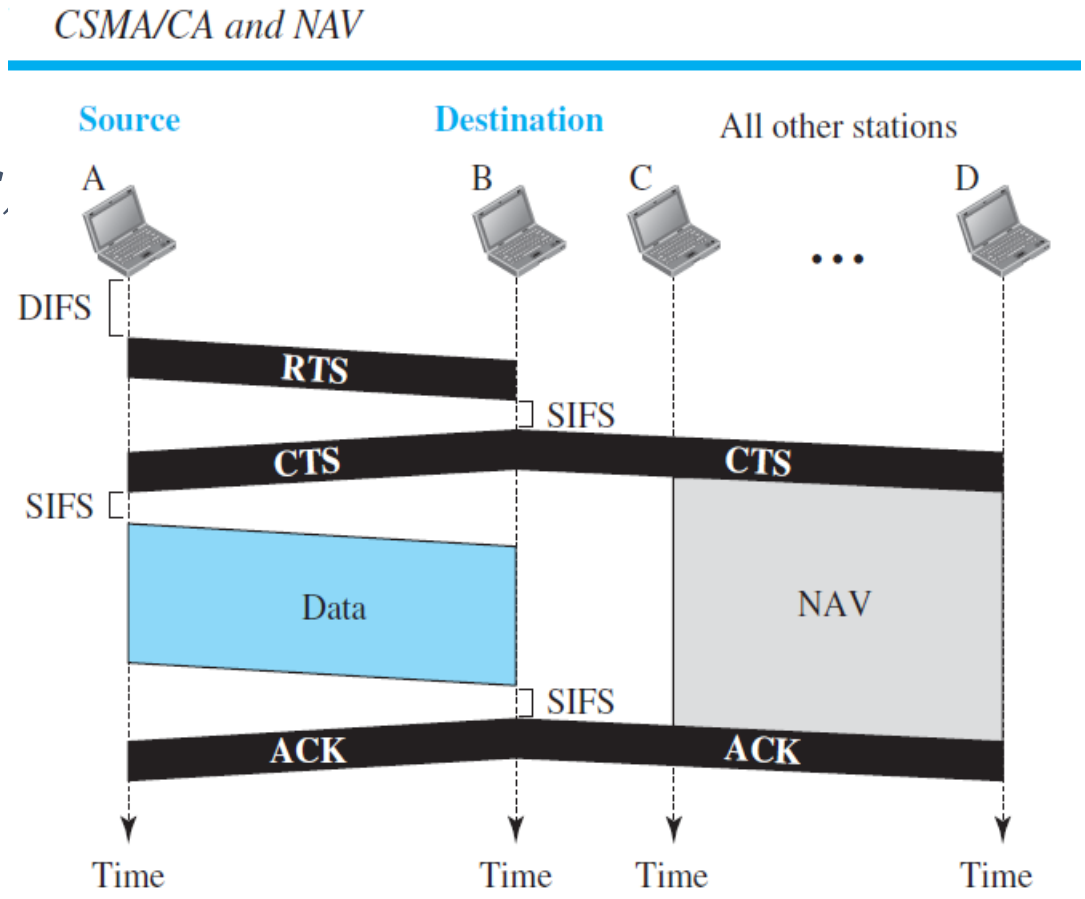
Acknowledgment:

- The positive acknowledgment and the time-out timer can help guarantee that the receiver has received the frame.


Legend
K: Number of attempts
T_B: Backoff time
IFS: Interframe Space
RTS: Request to send
CTS: Clear to send
K = 0
Channel free?
[false]
[true]
Carrier sense
Wait IFS
Contention window
Choose a random number R between 0 and $2^K - 1$ and use the Rth slot
Send RTS
Wait T_B seconds
Set a timer
[false]
CTS received before time-out?
[true]
Wait IFS
Send the frame
Transmission
Set a timer
[true]
K < limit ?
K = K + 1
[false]
ACK received before time-out?
[false]
[true]
Abort
Success

CSMA/ CA and NAV

DIFS: DCF interframe space
RTS: Request to Send
SIFS: Short interframe space (SIFS)
NAV: Network Allocation Vector



CONTROLLED ACCESS

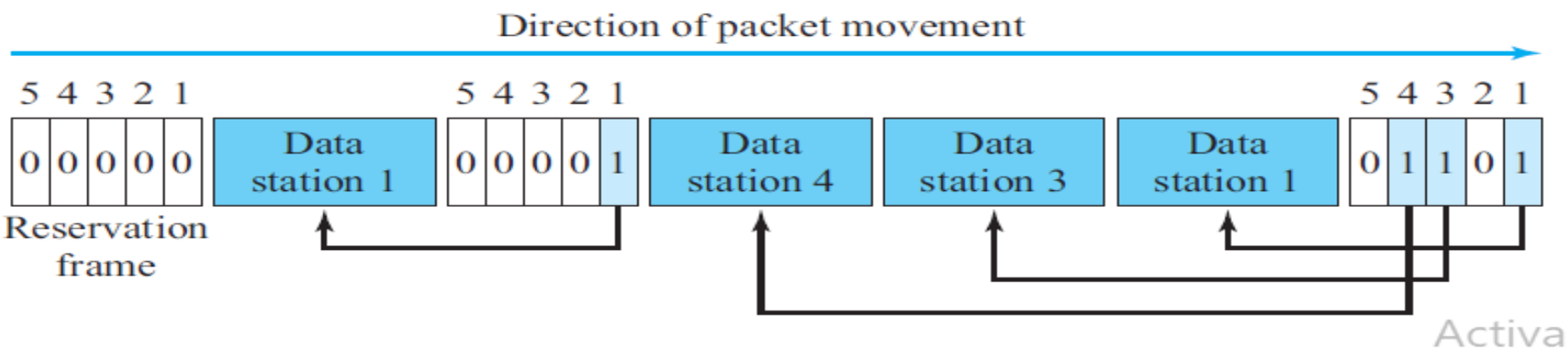
- In **controlled access,** the stations consult one another to find which station has the right to send.

- A station cannot send unless it has been authorized by other stations.

- Three common methods:
  - **Reservation**
  - **Polling**
  - **Token passing**

Reservation

- A station needs to make a **reservation** before sending data.
- **Time is divided into intervals.**
- In each interval, a reservation frame precedes the **data frames** sent in that interval.
- If there are **N stations** in the system, there are exactly **N reservation minislots** in the reservation frame.



Activa

Polling

- **Polling** works with topologies in which one device is designated as a ***primary station*** and the other devices are ***secondary stations*.**
- **Primary device** is the **initiator** of a session.
- **All data exchanges must be made through the primary device**.
- **Primary device controls the link;** the secondary devices follow its **instructions**.

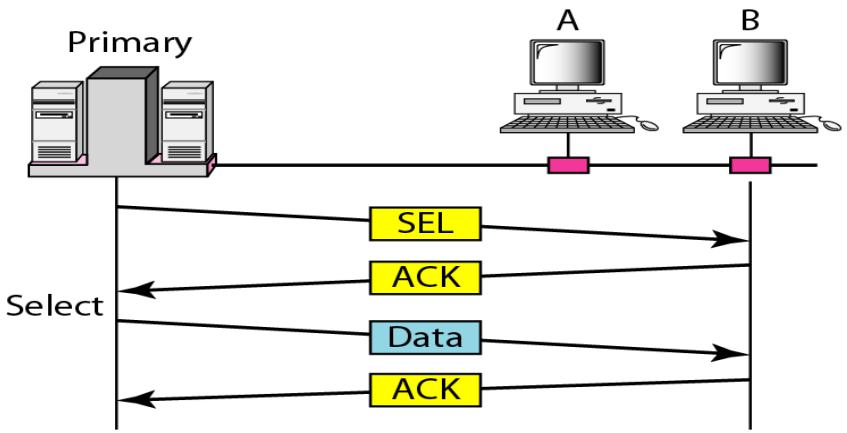
Select and poll functions in polling-access method

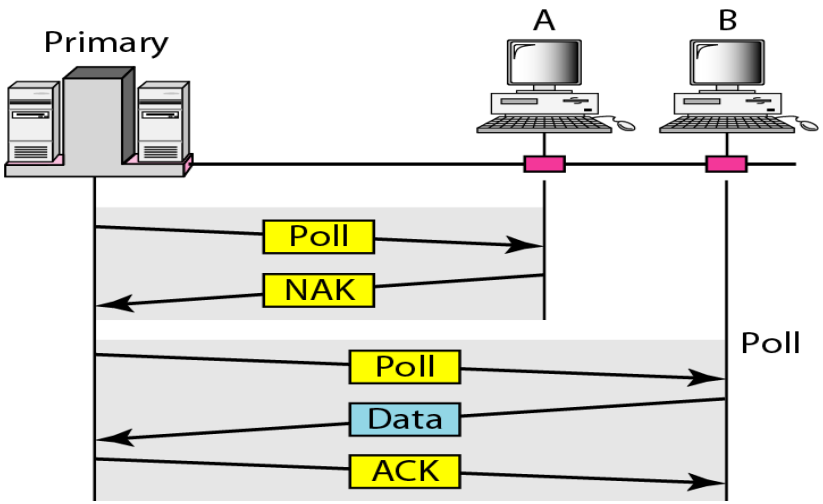
Select

The *select* function is used whenever the **primary device has something to send**.

Poll

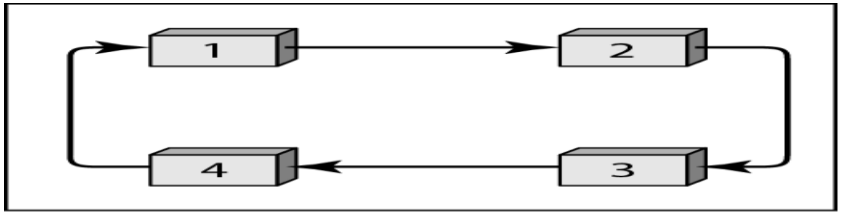
The *poll* function is used by the **primary device to solicit transmissions** from the **secondary devices**.



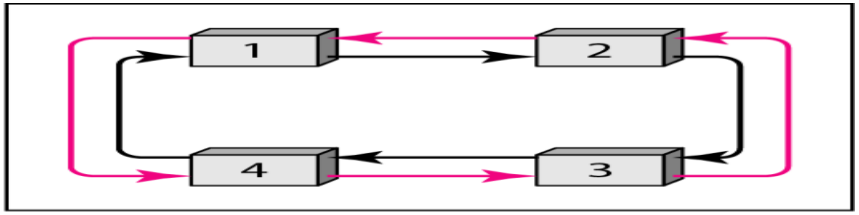


Token Passing

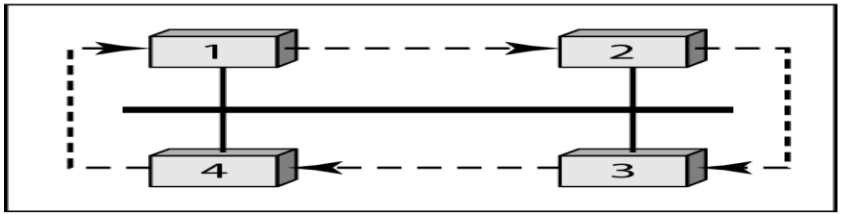
- The stations in a network are organized in **a logical ring.**
- For each station, there is **a predecessor and a successor.**
- The **right to this access** has been passed from the **predecessor to the current** station.
- The right will be passed to the **successor when the current station has no more data to send.**
- The RIGHT passed from by means of special packet called **"TOKEN".**



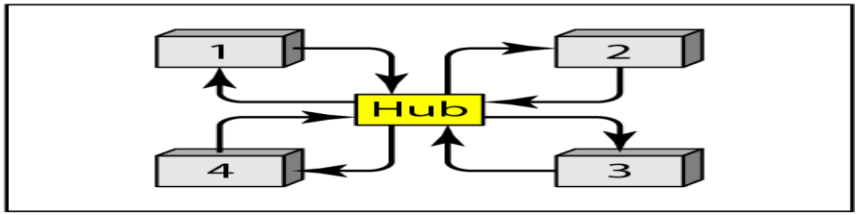
a. Physical ring



b. Dual ring



c. Bus ring



d. Star ring

Channelization

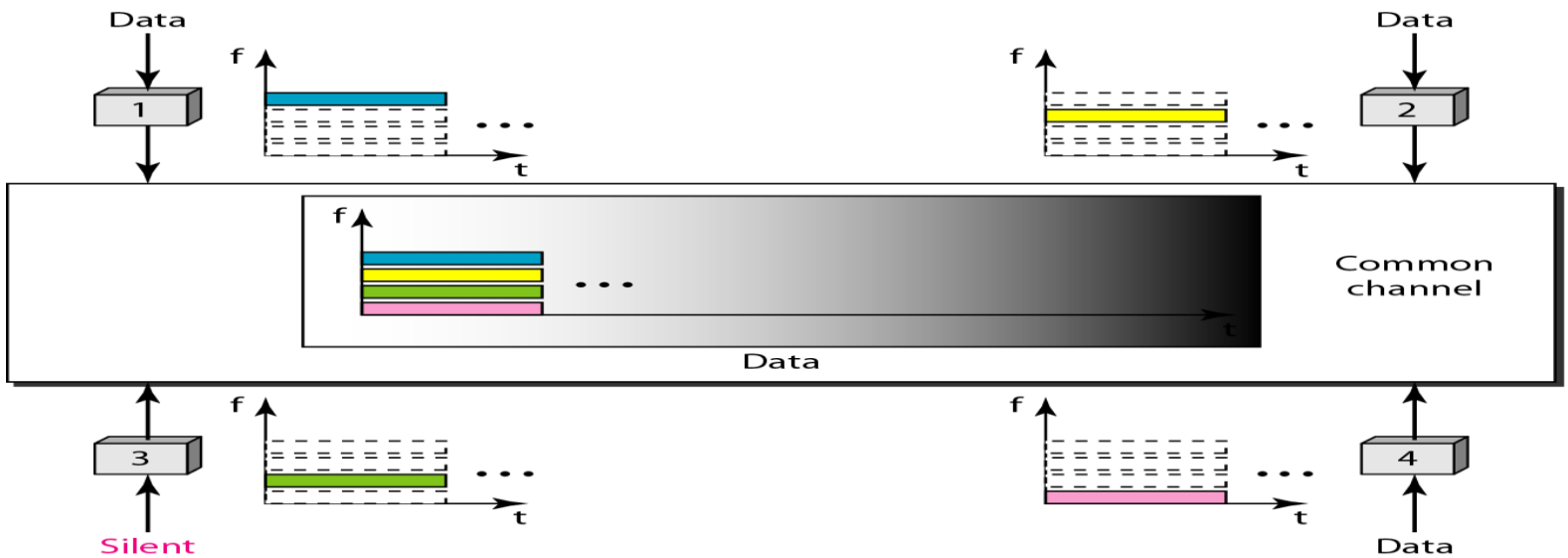
- **Channelization** is a multiple-access method in which the available bandwidth of a link is shared in **time**, **frequency,** or through **code**, among different stations.

- Similar to multiplexing

- Three schemes
  - **Frequency-Division Multiple Access (FDMA)**
  - **Time-Division Multiple Access (TDMA)**
  - **Code-Division Multiple Access (CDMA)**

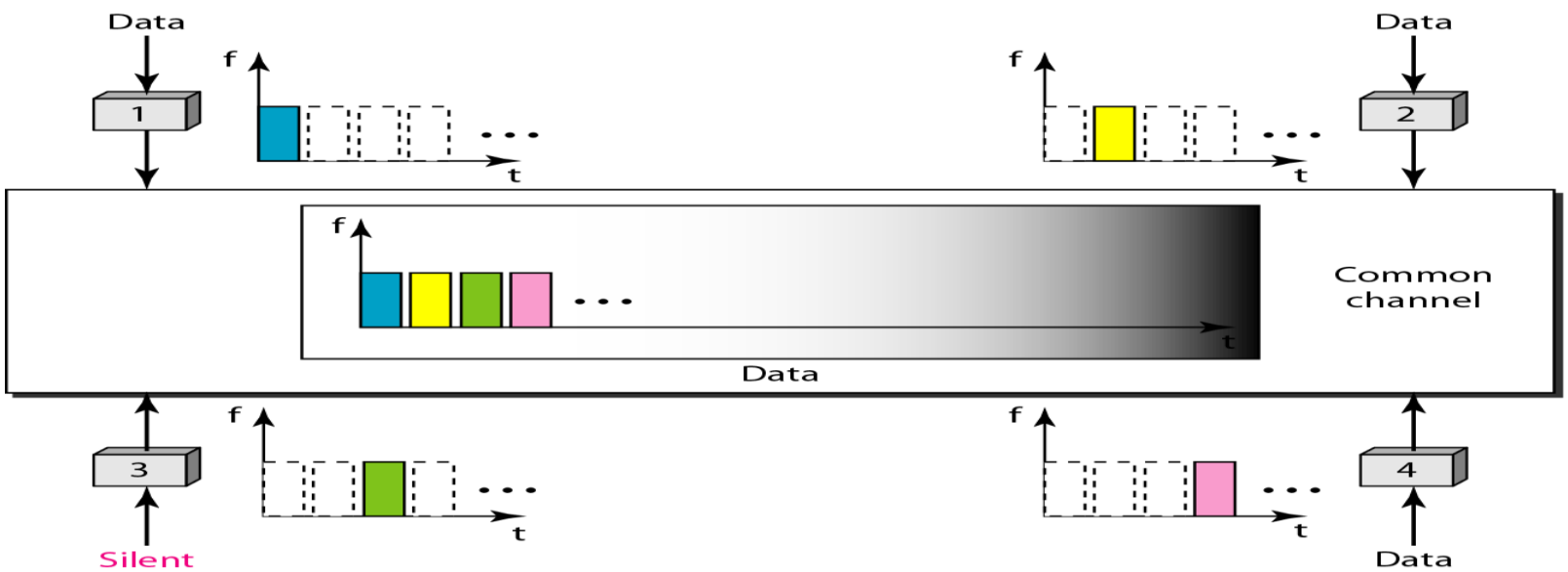
Frequency-Division Multiple Access (FDMA)

- Available bandwidth is divided into frequency bands.
- Each band is reserved for a specific station.
- Each station also uses a bandpass filter to confine the transmitter frequencies.



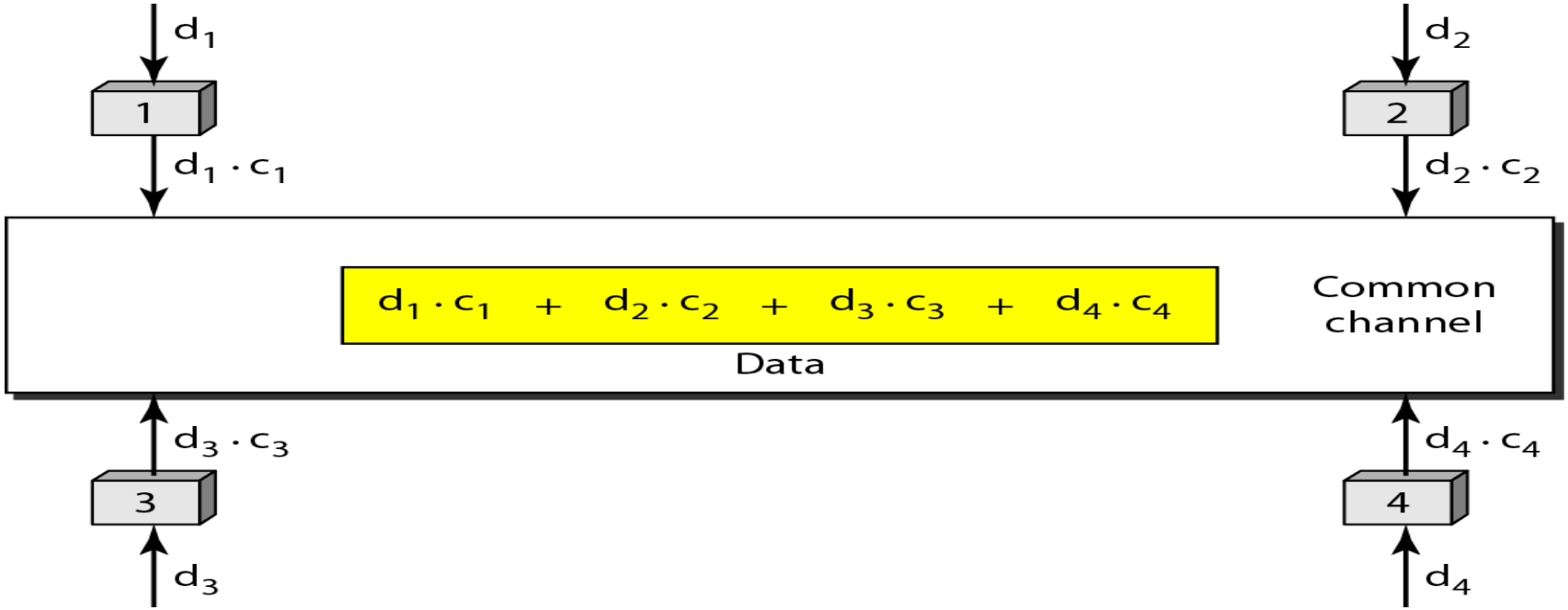
Time-Division Multiple Access (TDMA)

- Stations **share the bandwidth** of the channel in time.
- Each station is allocated a **time slot** during which it can send data.



Code-Division Multiple Access (CDMA)

- *One channel carries all transmissions at the same time*
- *Each channel is separated by code*



END OF MODULE 2